

Review mã nguồn toàn hệ thống VideoLevel

Từ circuit Groth16 đến H.264/CAVLC, API, blind extraction và nhánh Python

Tài liệu kỹ thuật tiếng Việt

26 tháng 9 năm 2026

Mục lục

1	Phạm vi và cách đọc bản review	2
2	Sơ đồ hoạt động của toàn hệ thống	2
3	Bản đồ module: thành phần nào sở hữu trách nhiệm nào	2
4	Lớp zero-knowledge: circuit và cầu nối snarkjs	3
4.1	Circuit thực sự chứng minh điều gì	3
4.2	Đóng gói message và proof	4
4.3	Tạo proof, dựng witness và public inputs	4
4.4	Serialize proof và biên dữ liệu	5
5	Native H.264/CAVLC: từ Annex-B đến bit candidate	6
5.1	Cắt elementary stream thành NAL unit	6
5.2	Giải mã IDR và tìm trailing-one sign offsets	6
5.3	Chọn candidate theo khóa và sửa sign bit	8
5.4	Đóng gói frame xác thực riêng của native route	9
5.5	Incremental encoder và decoder theo segment	10
6	Native CLI: process boundary và streaming I/O	11
6.1	Live embed	11
6.2	Live blind extraction	12
7	HTTP/WebSocket adapter và service API	13
7.1	Native adapter đầy đủ	13
7.2	HTTP job service	18
8	Python full-video route: cùng hệ thống, implementation khác	22
8.1	Orchestrator nhúng của Python	22
8.2	Verifier của Python route	26
8.3	Python H.264/CAVLC parser và safety layer	28
9	Trust, key policy và provenance: module hỗ trợ, không tự động cường chế	28
10	Contract giữa các bước: input/output và lỗi cần phân loại	29
11	Kết luận review: hệ thống hiện có và chưa có gì	29

1 Phạm vi và cách đọc bản review

Đây là tài liệu review implementation của hệ thống VideoLevel, không phải hướng dẫn chạy các script demo. Trọng tâm là các thành phần runtime hiện có: circuit và cầu nối Groth16; đường native C++ đọc/ghi H.264 CAVLC; đóng gói frame xác thực; HTTP/WebSocket; verifier/extractor; cùng nhánh Python full-video còn tồn tại.

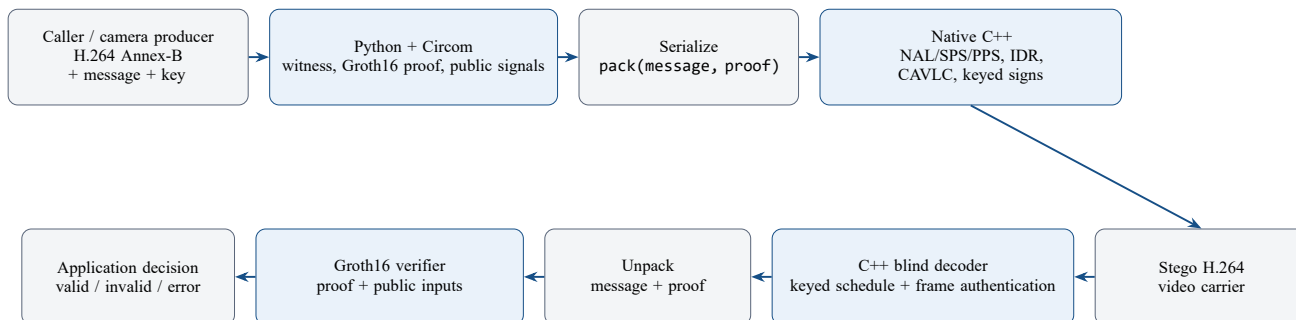
Các code listing được lấy trực tiếp từ source tree lúc biên soạn, có số dòng gốc để đối chiếu. Với mỗi hàm được đưa vào listing, toàn bộ thân hàm được hiển thị rồi mới nhận xét hợp đồng input/output, luồng điều khiển, dữ liệu nhạy cảm, lỗi có thể phát sinh và consumer tiếp theo. Các thư viện ngoài, mã sinh tự động, test và hàng nghìn dòng bảng VLC không được chép lại; chúng được phân tích qua ranh giới mà runtime gọi.

Phân biệt hai tuyến đang có trong repo là trọng tâm review:

- **Native keyed-blind route:** C++ phân tích Baseline/CAVLC, chọn sign-bit theo khóa, nhúng frame xác thực, rồi trích xuất không cần cover gốc. API native hiện gọi tuyến này.
- **Python legacy route:** Python phân tích cover, chọn vị trí an toàn, tái dựng bitstream và verifier thường cần cover/sidecar hoặc position map. Tuyến này vẫn có thể chạy nhưng không phải cùng implementation với native.

Các đoạn source được đọc như review code: đầu tiên xác định precondition, sau đó theo dữ liệu từng bước, cuối cùng kiểm tra kết quả có đúng với hợp đồng ở ranh giới kế tiếp hay không. Khi bảng hoặc code snippet nói "đầu ra", đó là đầu ra thực tế của implementation, không phải cam kết về mọi profile H.264.

2 Sơ đồ hoạt động của toàn hệ thống



Hình 1: Video là carrier. Nó không tự mang đầy đủ mọi cấu hình cần cho extraction và verification.

Luồng dữ liệu chuẩn ở mức khái niệm là:

$$\begin{aligned}
 m, k &\longrightarrow (\pi, \text{public}) \longrightarrow B = \text{pack}(m, \text{serialize}(\pi)), \\
 B, k &\longrightarrow V_{\text{stego}} = \text{embed}_{H.264}(B, k), \\
 V_{\text{stego}}, k &\longrightarrow (m, \pi) \longrightarrow \text{Verify}(\pi, \text{public}).
 \end{aligned}$$

Native channel không tạo Groth16 proof. Caller tạo proof-bearing bytes trước, rồi C++ xử lý chúng như payload opaque. Khóa CAVLC/HMAC cũng không được nhúng nguyên văn trong H.264; bên nhận phải có khóa qua cấu hình hoặc kênh cấp khóa riêng.

3 Bản đồ module: thành phần nào sở hữu trách nhiệm nào

Module	Trách nhiệm	Output được downstream dùng
circuits/payload_verify.circom	Ràng buộc quan hệ SHA-256 giữa payload hash, secret và commitment; kiểm tra miền độ dài.	Constraint system và public-signal contract cho proving key/verification key.
src/zk_proof.py	Tính witness/public signals, gọi snarkjs, serialize proof, đóng gói/tách payload.	Proof bytes 129 byte và blob [len][message][proof].

Module	Trách nhiệm	Output được downstream dùng
native/src/cavlc_stream.cpp	Annex-B, SPS/PPS, slice header, macroblock/CAVLC, candidate signs, keyed schedule, patch/recover.	Byte stream H.264 đã nhúng hoặc payload sau authenticated extraction.
native/tools/blind_bits.cpp	Process boundary/CLI: parse key/payload, nhận stream, gọi encoder/decoder, ghi output.	Exit status, video bytes hoặc payload bytes qua stdout.
src/api/native_handlers.py	HTTP/WebSocket adapter; đưa key an toàn vào stdin child và forward dữ liệu media.	HTTP jobs/artifact và live byte stream.
src/api/app.py	Request guard, auth token, upload limits, bounded job queue, status/artifact lifecycle.	HTTP protocol; không tự cài semantics của CAVLC hay circuit.
src/embedder.py, src/verifier.py	Nhánh Python full-video/sidecar: phân tích cover, safety filter, patch/reconstruct, extract, verify.	Video tái dựng, sidecars và VerifyResult.
src/key_policy.py, src/manifest.py, src/trust/	Certificate/manifest/provenance helpers phụ trợ.	Trust metadata nếu caller tích hợp; raw native key hiện không tự hết hạn.

4 Lớp zero-knowledge: circuit và cầu nối snarkjs

4.1 Circuit thực sự chứng minh điều gì

Circuit dùng ba public input: `payload_hash[256]`, `commitment[256]`, `payload_length`; secret key 256 bit là private witness. Quan hệ được ràng buộc là

$$h = \text{SHA256}(m), \quad c = \text{SHA256}(h||k), \quad 0 < |m| < 1,000,000.$$

Điểm quan trọng khi review: circuit nhận hash h làm public input, không nhận toàn bộ video hay chứng minh rằng video là nguyên gốc/camera-authentic. Thuộc tính media đó không nằm trong circuit hiện tại.

Listing 1: Circom source, dòng 1–76

```

1  pragma circom 2.0.0;
2
3  include "node_modules/circomlib/circuits/sha256/sha256.circom";
4  include "node_modules/circomlib/circuits/bitify.circom";
5  include "node_modules/circomlib/circuits/comparators.circom";
6
7  /*
8   * ZK-SNARK Circuit for Video Steganography Payload Verification
9   *
10  * Proves knowledge of a secret that produces a commitment to the payload
11  * without revealing the secret itself.
12  *
13  * Public inputs:
14  *   - payload_hash: SHA256 hash of the payload (256 bits)
15  *   - commitment: SHA256(payload_hash || secret) (256 bits)
16  *   - payload_length: Length of payload in bytes
17  *
18  * Private inputs:
19  *   - secret: Secret key (256 bits)
20  *
21  * Circuit verifies:
22  *   commitment = SHA256(payload_hash || secret)
23  */
24
25  template PayloadVerify() {
26    // Public inputs
27    signal input payload_hash[256]; // SHA256 hash of payload (256 bits)
28    signal input commitment[256]; // Expected commitment (256 bits)
29    signal input payload_length; // Payload length in bytes
30
31    // Private inputs
32    signal input secret[256]; // Secret key (256 bits)
33
34    // Intermediate signals
35    signal input_bits[512]; // payload_hash + secret (512 bits)
36
37    // Copy payload_hash to first 256 bits
38    for (var i = 0; i < 256; i++) {

```

```

39     input_bits[i] <== payload_hash[i];
40 }
41
42 // Copy secret to last 256 bits
43 for (var i = 0; i < 256; i++) {
44     input_bits[256 + i] <== secret[i];
45 }
46
47 // Compute SHA256(payload_hash || secret)
48 component sha = Sha256(512);
49 for (var i = 0; i < 512; i++) {
50     sha.in[i] <== input_bits[i];
51 }
52
53 // Store computed commitment in intermediate signal
54 signal computed_commitment[256];
55 for (var i = 0; i < 256; i++) {
56     computed_commitment[i] <== sha.out[i];
57 }
58
59 // Verify commitment matches computed value
60 for (var i = 0; i < 256; i++) {
61     commitment[i] === computed_commitment[i];
62 }
63
64 // Constrain the public length to  $0 < \text{length} < 1,000,000$  bytes.
65 // LessThan(20) range-checks both operands before comparing them.
66 component length_lt_limit = LessThan(20);
67 length_lt_limit.in[0] <== payload_length;
68 length_lt_limit.in[1] <== 1000000;
69 length_lt_limit.out === 1;
70
71 component length_is_zero = IsZero();
72 length_is_zero.in <== payload_length;
73 length_is_zero.out === 0;
74 }
75
76 component main {public [payload_hash, commitment, payload_length]} = PayloadVerify();

```

Nhận xét review. Đầu vào circuit là bit arrays đã được caller dựng. Hai vòng lặp đầu chỉ ghép public hash với private secret; Sha256(512) sinh commitment; toán tử === biến so sánh thành constraint. Nếu constraint sai, witness/proof không hợp lệ. LessThan(20) và IsZero() thêm range/nonzero cho length. Hàm main quyết định thứ tự public signals; thay đổi thứ tự/constraint buộc phải build lại artifacts proving/verification. Không nên diễn giải proof này thành chứng nhận video chưa bị transcode: circuit không hash bitstream video.

4.2 Đóng gói message và proof

Listing 2: Python source, dòng 155–174

```

155 →→→→→→→→→→
156 def pack(message_bytes: bytes, proof_bytes: bytes) -> bytes:
157     """
158     Pack message + proof → single blob:
159     [4 bytes big-endian: len(message)][message][proof (129 bytes)]
160     """
161     assert len(proof_bytes) == PROOF_SIZE_BYTES
162     return struct.pack(">I", len(message_bytes)) + message_bytes + proof_bytes
163
164
165 def unpack(blob: bytes) -> Tuple[bytes, bytes]:
166     """Unpack blob → (message_bytes, proof_bytes). Raises ValueError on bad data."""
167     if len(blob) < 4:
168         raise ValueError(f"Blob too short: {len(blob)} bytes")
169     msg_len = struct.unpack(">I", blob[:4])[0]
170     expected = 4 + msg_len + PROOF_SIZE_BYTES
171     if len(blob) < expected:
172         raise ValueError(f"Blob too short: expected {expected}, got {len(blob)}")
173     return blob[4: 4 + msg_len], blob[4 + msg_len: 4 + msg_len + PROOF_SIZE_BYTES]

```

Nhận xét review. pack() viết length của message thành 4-byte big-endian, sau đó nối message và proof đã serialize. Hàm không tự kiểm tra proof cryptographically; nó chỉ đóng gói bytes. unpack() đọc length và xác nhận độ dài blob đúng chính xác bằng $4 + |m| + |\pi|$, rồi cắt hai phần. Bởi vậy, caller ở bước sau phải gọi verifier Groth16; unpack thành công chỉ chứng minh format không sai. Ở format hiện tại proof compressed có 129 byte; length field chỉ mô tả message, không mô tả proof.

4.3 Tạo proof, dựng witness và public inputs

Listing 3: Python source, dòng 214–246

```

214 →+++++
215 def generate_proof_for_payload(
216     self, payload_bytes: bytes, secret_key: bytes
217 ) -> Tuple[dict, dict]:
218     """
219     Full pipeline: payload → witness → Groth16 proof.
220
221     Returns:
222     (proof_dict, public_dict) – snarkjs format dicts.
223     """
224     self._check_node_available()
225     logger.info("[ZK] Computing commitment...")
226     circuit_input = self._build_circuit_input(payload_bytes, secret_key)
227     logger.info("[ZK] Computing witness (node.js wasm)...")
228     witness_path = self._compute_witness(circuit_input)
229     logger.info("[ZK] Generating Groth16 proof (snarkjs)...")
230     return self._snarkjs_prove(witness_path)
231
232 def verify(self, proof_dict: dict, public_dict: dict) -> bool:
233     """Verify a Groth16 proof using snarkjs. Returns True if valid."""
234     self._check_node_available()
235     logger.info("[ZK] Verifying Groth16 proof...")
236     return self._snarkjs_verify(proof_dict, public_dict)
237
238 def verify_proof_for_payload(
239     self, proof_dict: dict, payload_bytes: bytes, secret_key: bytes,
240 ) -> bool:
241     """Verify a proof against the public signals derived from payload and key."""
242     if not isinstance(payload_bytes, bytes) or not payload_bytes:
243         raise ValueError("payload_bytes must be non-empty bytes")
244     if not isinstance(secret_key, bytes) or len(secret_key) != 32:
245         raise ValueError("secret_key must be exactly 32 bytes")
246     public_signals = self._build_public_signals(payload_bytes, secret_key)
247     return self.verify(proof_dict, public_signals)

```

Nhận xét review. `generate_proof_for_payload(payload, key)` kiểm tra Node, dựng input circuit, tính witness bằng WASM rồi gọi `snarkjs prove`. Output là proof dictionary và public dictionary. `verify_proof_for_payload()` tái-dựng public inputs từ chính bytes payload và key rồi gọi verifier. Tên biến `payload_bytes` vì thế có nghĩa là message đang được chứng minh, không phải video carrier hay outer packed blob. Trong hệ thống, caller phải đưa cùng message/key khi tái dựng public signals.

Listing 4: Python source, dòng 269–286

```

269 →+++++
270 def _build_circuit_input(self, payload_bytes: bytes, secret_key: bytes) -> dict:
271     assert len(secret_key) == 32, "secret_key must be exactly 32 bytes"
272     payload_hash_bytes = hashlib.sha256(payload_bytes).digest()
273     commitment_bytes = hashlib.sha256(payload_hash_bytes + secret_key).digest()
274     return {
275         "payload_hash": self._bytes_to_bits(payload_hash_bytes),
276         "commitment": self._bytes_to_bits(commitment_bytes),
277         "payload_length": len(payload_bytes),
278         "secret": self._bytes_to_bits(secret_key),
279     }
280
281 def _build_public_signals(self, payload_bytes: bytes, secret_key: bytes) -> list:
282     payload_hash_bytes = hashlib.sha256(payload_bytes).digest()
283     commitment_bytes = hashlib.sha256(payload_hash_bytes + secret_key).digest()
284     return (self._bytes_to_bits(payload_hash_bytes) +
285             self._bytes_to_bits(commitment_bytes) +
286             [str(len(payload_bytes))])

```

Nhận xét review. Hai hàm này là nơi semantics cryptographic được chuyển sang witness JSON/public vector. `_build_circuit_input()` có secret bits nên output là dữ liệu nhạy cảm; downstream là witness generator, không phải API public response. `_build_public_signals()` không đưa secret trực tiếp vào public vector, mà đưa hash, commitment và length. Tuy vậy code wrapper vẫn cần key để tính commitment. Nếu muốn verifier độc lập không biết secret, protocol phải cung cấp commitment/public signals đáng tin cậy cùng proof; implementation convenience path hiện nhận key để tính lại.

4.4 Serialize proof và biên dữ liệu

Listing 5: Python source, dòng 64–93

```

64 →+++++

```

```

65 def proof_to_bytes(proof_dict: dict) -> bytes:
66     """
67     Serialize a snarkjs Groth16 proof dict → 129-byte point compressed binary.
68     """
69     def _to32(val_str: str) -> bytes:
70         return int(val_str).to_bytes(32, "big")
71
72     pi_a = proof_dict["pi_a"]
73     pi_b = proof_dict["pi_b"]
74     pi_c = proof_dict["pi_c"]
75
76     # Signs for Y coordinates (0 if even, 1 if odd)
77     y_a_sign = int(pi_a[1]) & 1
78     y_b0, y_b1 = int(pi_b[1][0]), int(pi_b[1][1])
79     y_b_sign = (y_b0 & 1) if y_b0 != 0 else (y_b1 & 1)
80     y_c_sign = int(pi_c[1]) & 1
81
82     flags = (y_a_sign << 2) | (y_b_sign << 1) | y_c_sign
83
84     blob = b"".join([
85         _to32(pi_a[0]),      # pi_a.x
86         _to32(pi_b[0][0]),   # pi_b.x[0]
87         _to32(pi_b[0][1]),   # pi_b.x[1]
88         _to32(pi_c[0]),      # pi_c.x
89         flags.to_bytes(1, "big")
90     ])
91     assert len(blob) == PROOF_SIZE_BYTES
92     return blob

```

Nhận xét review. Serializer ánh xạ các tọa độ proof Groth16 sang dạng nén cố định. Đây là biên định dạng giữa JSON snarkjs và video payload. Rà soát version/canonical encoding ở đây quan trọng: mọi khác biệt byte-order, field modulus hoặc độ dài sẽ làm proof không round-trip. Đừng sửa serializer một phía; phải đồng thời cập nhật deserialize, test vectors, payload capacity và verifier.

5 Native H.264/CAVLC: từ Annex-B đến bit candidate

5.1 Cắt elementary stream thành NAL unit

Listing 6: C++ source, dòng 1136–1164

```

1136 std::vector<AnnexBNalUnit> split_annex_b(const std::vector<std::uint8_t>& annex_b) {
1137     std::vector<AnnexBNalUnit> units;
1138     std::size_t cursor = 0;
1139     while (cursor < annex_b.size()) {
1140         const auto marker_length = start_code_length_at(annex_b, cursor);
1141         if (marker_length == 0) {
1142             ++cursor;
1143             continue;
1144         }
1145         const auto start = cursor;
1146         const auto header_offset = cursor + marker_length;
1147         if (header_offset >= annex_b.size()) break;
1148         cursor = header_offset + 1;
1149         while (cursor < annex_b.size() && start_code_length_at(annex_b, cursor) == 0) ++cursor;
1150
1151         const auto header = annex_b[header_offset];
1152         AnnexBNalUnit unit;
1153         unit.start_offset = start;
1154         unit.start_code_size = marker_length;
1155         unit.forbidden_zero_bit = static_cast<std::uint8_t>((header >> 7) & 0x01);
1156         unit.nal_ref_idc = static_cast<std::uint8_t>((header >> 5) & 0x03);
1157         unit.nal_unit_type = static_cast<std::uint8_t>(header & 0x1f);
1158         unit.payload.assign(annex_b.begin() + static_cast<std::ptrdiff_t>(header_offset + 1),
1159                             annex_b.begin() + static_cast<std::ptrdiff_t>(cursor));
1160         units.push_back(std::move(unit));
1161     }
1162     return units;
1163 }

```

Nhận xét review. `split_annex_b()` quét start code 3/4 byte, phân vùng input thành NAL, giữ vị trí/độ dài/header để serializer patch lại đúng byte range. Output vector NAL downstream được dùng bởi SPS/PPS/IDR decoder. Đây là byte-level parser, không phải FFmpeg decode frame thành pixel. Invariant cần bảo toàn: payload EBSP được chuyển RBSP chỉ ở lúc parse syntax; khi ghi lại phải tái chèn emulation-prevention bytes để byte sequence không giả start code.

5.2 Giải mã IDR và tìm trailing-one sign offsets

Listing 7: C++ source, dòng 1312–1388

```

1312 std::vector<CavlcDecodedIdrSlice> decode_baseline_i_idr_slices(
1313     const std::vector<std::uint8_t>& annex_b) {
1314     std::unordered_map<std::uint32_t, H264BaselineSps> sps_by_id;
1315     std::unordered_map<std::uint32_t, H264BaselinePps> pps_by_id;
1316     std::vector<CavlcDecodedIdrSlice> decoded_slices;
1317     const auto units = split_annex_b(annex_b);
1318     for (std::size_t nal_index = 0; nal_index < units.size(); ++nal_index) {
1319         const auto& nal = units[nal_index];
1320         const auto rbsp = nal.rbsp();
1321         if (nal.nal_unit_type == 7U) {
1322             const auto sps = parse_baseline_sps(rbsp);
1323             sps_by_id.insert_or_assign(sps.sequence_parameter_set_id, sps);
1324             continue;
1325         }
1326         if (nal.nal_unit_type == 8U) {
1327             const auto pps = parse_baseline_pps(rbsp);
1328             pps_by_id.insert_or_assign(pps.pic_parameter_set_id, pps);
1329             continue;
1330         }
1331         if (!nal.is_idr()) continue;
1332
1333         RbspBitReader prefix_reader(rbsp);
1334         static_cast<void>(prefix_reader.read_ue()); // first_mb_in_slice
1335         static_cast<void>(prefix_reader.read_ue()); // slice_type
1336         const auto pps_id = prefix_reader.read_ue();
1337         const auto pps_it = pps_by_id.find(pps_id);
1338         if (pps_it == pps_by_id.end()) {
1339             throw std::invalid_argument("IDR references an unavailable PPS");
1340         }
1341         const auto sps_it = sps_by_id.find(pps_it->second.sequence_parameter_set_id);
1342         if (sps_it == sps_by_id.end()) {
1343             throw std::invalid_argument("PPS references an unavailable SPS");
1344         }
1345         auto decoded = decode_baseline_i_idr_slice(rbsp, sps_it->second, pps_it->second);
1346         decoded.nal_index = nal_index;
1347         decoded_slices.push_back(std::move(decoded));
1348     }
1349     return decoded_slices;
1350 }
1351
1352 std::vector<CavlcSignCandidate> collect_cavlc_trailing_one_sign_candidates(
1353     const std::vector<CavlcDecodedIdrSlice>& slices) {
1354     std::vector<CavlcSignCandidate> candidates;
1355     for (const auto& slice : slices) {
1356         for (const auto& macroblock : slice.macroblocks) {
1357             const auto append = [&](const CavlcDecodedLumaBlock& block,
1358                                     const CavlcResidualCategory category,
1359                                     const std::uint8_t block_index) {
1360                 if (block.token.sign_bit_offsets.empty()) return;
1361                 candidates.push_back(CavlcSignCandidate{
1362                     slice.nal_index,
1363                     macroblock.address,
1364                     category,
1365                     block_index,
1366                     block.token.sign_bit_offsets.front(),
1367                 });
1368             };
1369             if (macroblock.header.mb_type >= 1U && macroblock.header.mb_type <= 24U) {
1370                 append(macroblock.luma_dc, CavlcResidualCategory::LumaDc, 0U);
1371             }
1372             for (std::size_t block_index = 0; block_index < macroblock.luma.blocks.size(); ++block_index) {
1373                 append(macroblock.luma.blocks[block_index], CavlcResidualCategory::Luma4x4,
1374                     static_cast<std::uint8_t>(block_index));
1375             }
1376             for (std::size_t component = 0; component < macroblock.chroma_dc.size(); ++component) {
1377                 append(macroblock.chroma_dc[component], CavlcResidualCategory::ChromaDc,
1378                     static_cast<std::uint8_t>(component));
1379             }
1380             for (std::size_t block_index = 0; block_index < macroblock.chroma_ac.size(); ++block_index) {
1381                 append(macroblock.chroma_ac[block_index], CavlcResidualCategory::ChromaAc,
1382                     static_cast<std::uint8_t>(block_index));
1383             }
1384         }
1385     }
1386     return candidates;
1387 }

```

Nhận xét review. Listing này gồm trọn hai hàm: hàm đầu duy trì context SPS/PPS gần nhất, chọn NAL IDR phù hợp rồi giải mã I-slice; hàm sau đi qua residual blocks và phát candidate theo sign bit của trailing one. Candidate lưu NAL index, macroblock/block identity và RBSP bit offset. Nó là địa chỉ trong syntax stream, không phải tọa độ pixel. Mọi consumer phải dùng chính parser này khi embed và blind-extract để lịch hai đầu trùng nhau.

5.3 Chọn candidate theo khóa và sửa sign bit

Listing 8: C++ source, dòng 1552–1638

```

1552 std::vector<CavlcSignCandidate> select_keyed_cavlc_sign_candidates(
1553     const std::vector<CavlcSignCandidate>& candidates,
1554     const std::vector<std::uint8_t>& secret_key,
1555     const std::size_t required_bits) {
1556     if (secret_key.size() != 32U) throw std::invalid_argument("blind schedule key must be exactly 32 bytes");
1557     if (required_bits > candidates.size()) throw std::invalid_argument("blind schedule capacity is insufficient");
1558     struct ScoredCandidate { std::array<std::uint8_t, 32> score; std::string identity; CavlcSignCandidate candidate; };
1559     std::vector<ScoredCandidate> scored;
1560     scored.reserve(candidates.size());
1561     std::unordered_set<std::string> identities;
1562     std::unordered_set<std::string> physical_targets;
1563     for (const auto& candidate : candidates) {
1564         validate_blind_schedule_candidate(candidate);
1565         auto identity = serialize_cavlc_sign_candidate(candidate);
1566         if (!identities.insert(identity).second) {
1567             throw std::invalid_argument("blind schedule candidate identity is duplicated");
1568         }
1569         const auto physical_target = std::to_string(candidate.nal_index) + ":" +
1570             std::to_string(candidate.rbsp_bit_offset);
1571         if (!physical_targets.insert(physical_target).second) {
1572             throw std::invalid_argument("blind schedule candidate patch target is duplicated");
1573         }
1574         scored.push_back({score_keyed_cavlc_sign_candidate(candidate, secret_key), std::move(identity), candidate});
1575     }
1576     std::sort(scored.begin(), scored.end(), [](const auto& left, const auto& right) {
1577         return left.score != right.score ? left.score < right.score : left.identity < right.identity;
1578     });
1579     std::vector<CavlcSignCandidate> selected;
1580     selected.reserve(required_bits);
1581     for (std::size_t index = 0; index < required_bits; ++index) selected.push_back(scored[index].candidate);
1582     return selected;
1583 }
1584
1585 std::vector<std::uint8_t> embed_keyed_cavlc_sign_bits(
1586     const std::vector<std::uint8_t>& annex_b,
1587     const std::vector<std::uint8_t>& secret_key,
1588     const std::vector<std::uint8_t>& payload_bits) {
1589     for (const auto bit : payload_bits) {
1590         if (bit > 1U) throw std::invalid_argument("CAVLC payload bits must be zero or one");
1591     }
1592     const auto candidates = collect_cavlc_trailing_one_sign_candidates(
1593         decode_baseline_i_idr_slices(annex_b));
1594     const auto selected = select_keyed_cavlc_sign_candidates(
1595         candidates, secret_key, payload_bits.size());
1596     const auto units = split_annex_b(annex_b);
1597     std::vector<std::vector<FixedLengthBitPatch>> patches_by_nal(units.size());
1598     for (std::size_t index = 0; index < selected.size(); ++index) {
1599         const auto& candidate = selected[index];
1600         patches_by_nal[candidate.nal_index].push_back(
1601             {candidate.rbsp_bit_offset, {payload_bits[index]}});
1602     }
1603     std::vector<AnnexBRbspPatchPlan> plan;
1604     plan.reserve(units.size());
1605     for (std::size_t nal_index = 0; nal_index < patches_by_nal.size(); ++nal_index) {
1606         if (!patches_by_nal[nal_index].empty()) {
1607             plan.push_back({nal_index, std::move(patches_by_nal[nal_index])});
1608         }
1609     }
1610     return patch_annex_b_rbsp_plan(annex_b, plan);
1611 }
1612
1613 std::vector<std::uint8_t> extract_keyed_cavlc_sign_bits(
1614     const std::vector<std::uint8_t>& annex_b,
1615     const std::vector<std::uint8_t>& secret_key,
1616     const std::size_t payload_bit_count) {
1617     const auto candidates = collect_cavlc_trailing_one_sign_candidates(
1618         decode_baseline_i_idr_slices(annex_b));
1619     const auto selected = select_keyed_cavlc_sign_candidates(
1620         candidates, secret_key, payload_bit_count);
1621     const auto units = split_annex_b(annex_b);
1622     std::vector<std::uint8_t> payload_bits;
1623     payload_bits.reserve(selected.size());
1624     for (const auto& candidate : selected) {
1625         if (candidate.nal_index >= units.size()) {
1626             throw std::invalid_argument("blind schedule candidate NAL is outside the segment");
1627         }
1628         const auto rbsp = units[candidate.nal_index].rbsp();
1629         if (candidate.rbsp_bit_offset >= rbsp.size() * 8U) {
1630             throw std::invalid_argument("blind schedule candidate bit is outside RBSP");
1631         }
1632         payload_bits.push_back(static_cast<std::uint8_t>(
1633             (rbsp[candidate.rbsp_bit_offset / 8U] >>
1634              (7U - candidate.rbsp_bit_offset % 8U)) & 1U));
1635     }
1636     return payload_bits;
1637 }

```

Nhận xét review. Đoạn source gồm đầy đủ `select_keyed_cavlc_sign_candidates()`, `embed_keyed_cavlc_sign_bits()` và `extract_keyed_cavlc_sign_bits()`. Selector tính điểm keyed cho từng định danh candidate, sắp thứ tự ổn định và chọn đủ số bit yêu cầu. Embed/extract tái dùng cùng selector; embed đảo đúng sign bit ở RBSP offset rồi assemble Annex-B. Đây là blind synchronization contract. Việc đổi serialization candidate hoặc domain HMAC làm mất tương thích hai phía.

5.4 Đóng gói frame xác thực riêng của native route

Listing 9: C++ source, dòng 1639–1717

```

1639 std::vector<std::uint8_t> pack_authenticated_cavlc_frame(
1640     const std::vector<std::uint8_t>& payload,
1641     const std::vector<std::uint8_t>& secret_key) {
1642     constexpr std::uint8_t frame_version = 1;
1643     constexpr std::size_t tag_bytes = 16;
1644     if (payload.size() > std::numeric_limits<std::uint16_t>::max()) {
1645         throw std::invalid_argument("authenticated payload exceeds 65535 bytes");
1646     }
1647     std::vector<std::uint8_t> frame;
1648     frame.reserve(3U + payload.size() + tag_bytes);
1649     frame.push_back(frame_version);
1650     frame.push_back(static_cast<std::uint8_t>(payload.size() >> 8U));
1651     frame.push_back(static_cast<std::uint8_t>(payload.size()));
1652     frame.insert(frame.end(), payload.begin(), payload.end());
1653     const auto tag = authenticated_frame_tag(secret_key, frame);
1654     frame.insert(frame.end(), tag.begin(), tag.begin() + static_cast<std::ptrdiff_t>(tag_bytes));
1655     return frame;
1656 }
1657
1658 std::vector<std::uint8_t> unpack_authenticated_cavlc_frame(
1659     const std::vector<std::uint8_t>& frame,
1660     const std::vector<std::uint8_t>& secret_key,
1661     const std::size_t maximum_payload_bytes) {
1662     constexpr std::uint8_t frame_version = 1;
1663     constexpr std::size_t header_bytes = 3;
1664     constexpr std::size_t tag_bytes = 16;
1665     if (maximum_payload_bytes > std::numeric_limits<std::uint16_t>::max()) {
1666         throw std::invalid_argument("authenticated payload maximum exceeds 65535 bytes");
1667     }
1668     if (frame.size() < header_bytes + tag_bytes || frame[0] != frame_version) {
1669         throw std::invalid_argument("authenticated CAVLC frame version is invalid");
1670     }
1671     const auto payload_size = (static_cast<std::size_t>(frame[1]) << 8U) | frame[2];
1672     if (payload_size > maximum_payload_bytes || frame.size() != header_bytes + payload_size + tag_bytes) {
1673         throw std::invalid_argument("authenticated CAVLC frame length is invalid");
1674     }
1675     const auto authenticated_size = header_bytes + payload_size;
1676     std::vector<std::uint8_t> authenticated_data(
1677         frame.begin(), frame.begin() + static_cast<std::ptrdiff_t>(authenticated_size));
1678     const auto expected_tag = authenticated_frame_tag(secret_key, authenticated_data);
1679     std::uint8_t difference = 0;
1680     for (std::size_t index = 0; index < tag_bytes; ++index) {
1681         difference |= static_cast<std::uint8_t>(expected_tag[index] ^ frame[authenticated_size + index]);
1682     }
1683     if (difference != 0U) throw std::invalid_argument("authenticated CAVLC frame tag is invalid");
1684     return {frame.begin() + static_cast<std::ptrdiff_t>(header_bytes),
1685         frame.begin() + static_cast<std::ptrdiff_t>(authenticated_size)};
1686 }
1687
1688 std::vector<std::uint8_t> embed_authenticated_cavlc_payload(
1689     const std::vector<std::uint8_t>& annex_b,
1690     const std::vector<std::uint8_t>& secret_key,
1691     const std::vector<std::uint8_t>& payload) {
1692     return embed_keyed_cavlc_sign_bits(
1693         annex_b, secret_key, bytes_to_msb_bits(pack_authenticated_cavlc_frame(payload, secret_key)));
1694 }
1695
1696 std::vector<std::uint8_t> extract_authenticated_cavlc_payload(
1697     const std::vector<std::uint8_t>& annex_b,
1698     const std::vector<std::uint8_t>& secret_key,
1699     const std::size_t maximum_payload_bytes) {
1700     constexpr std::uint8_t frame_version = 1;
1701     constexpr std::size_t header_bytes = 3;
1702     constexpr std::size_t tag_bytes = 16;
1703     if (maximum_payload_bytes > std::numeric_limits<std::uint16_t>::max()) {
1704         throw std::invalid_argument("authenticated payload maximum exceeds 65535 bytes");
1705     }
1706     const auto header = msb_bits_to_bytes(extract_keyed_cavlc_sign_bits(
1707         annex_b, secret_key, header_bytes * 8U));
1708     if (header.size() != header_bytes || header[0] != frame_version) {
1709         throw std::invalid_argument("authenticated CAVLC frame version is invalid");
1710     }
1711     const auto payload_size = (static_cast<std::size_t>(header[1]) << 8U) | header[2];
1712     if (payload_size > maximum_payload_bytes) {
1713         throw std::invalid_argument("authenticated CAVLC frame length exceeds configured maximum");
1714     }

```

```

1714 }
1715 const auto full_frame = msb_bits_to_bytes(extract_keyed_cavlc_sign_bits(
1716     annex_b, secret_key, (header_bytes + payload_size + tag_bytes) * 8U));
1717 return unpack_authenticated_cavlc_frame(full_frame, secret_key, maximum_payload_bytes);

```

Nhận xét review. Các hàm đầy đủ trong listing gồm pack/unpack authenticated frame và payload adapter. Native frame có version, length, payload và tag HMAC; overhead hiện tại 19 byte. Tag bảo vệ payload/framing dưới khóa chia sẻ; nó không thay proof Groth16. Nếu tag sai, extractor phải fail-closed trước khi trả payload cho parser của ZK. Ngược lại, tag hợp lệ chỉ chứng minh người biết key tạo frame, không chứng minh circuit statement đúng.

5.5 Incremental encoder và decoder theo segment

Listing 10: C++ source, dòng 1834–1863

```

1834 AuthenticatedCavlcStreamSegment AuthenticatedCavlcStreamEncoder::process_segment(
1835     const std::vector<std::uint8_t>& annex_b_segment) {
1836     if (complete()) return {annex_b_segment, 0U, 0U, true};
1837     auto prepared = prepare_cavlc_stream_segment(annex_b_segment, parameter_sets_);
1838     parameter_sets_ = std::move(prepared.parameter_sets_);
1839     const auto& candidates = prepared.candidates;
1840     const auto segment_capacity = std::min(maximum_bits_per_segment_, candidates.size());
1841     if (segment_capacity == 0U) return {annex_b_segment, 0U, 0U, false};
1842
1843     std::vector<std::uint8_t> segment_bits(segment_capacity, 0U);
1844     const auto bits_embedded = std::min(segment_capacity, remaining_bits());
1845     std::copy_n(frame_bits_.begin() + static_cast<std::ptrdiff_t>(next_bit_), bits_embedded,
1846         segment_bits.begin());
1847     const auto selected = select_keyed_cavlc_sign_candidates(candidates, secret_key_, segment_capacity);
1848     auto units = split_annex_b(prepared.analysis_input);
1849     std::vector<std::vector<FixedLengthBitPatch>> patches_by_nal(units.size());
1850     for (std::size_t index = 0; index < selected.size(); ++index) {
1851         patches_by_nal[selected[index].nal_index].push_back(
1852             {selected[index].rbsp_bit_offset, {segment_bits[index]}});
1853     }
1854     std::vector<AnnexBRbspPatchPlan> plan;
1855     for (std::size_t index = 0; index < patches_by_nal.size(); ++index) {
1856         if (!patches_by_nal[index].empty()) plan.push_back({index, std::move(patches_by_nal[index])});
1857     }
1858     auto patched = patch_annex_b_rbsp_plan(prepared.analysis_input, plan);
1859     auto output = remove_stream_context_prefix(patched, prepared.context_nal_count);
1860     next_bit_ += bits_embedded;
1861     return {std::move(output), segment_capacity, bits_embedded, complete()};
1862 }

```

Nhận xét review. `AuthenticatedCavlcStreamEncoder::process_segment()` nhận segment Annex-B, ghép SPS/PPS context nếu cần, phân tích slice/candidate, tính capacity segment, lấy số bit cần thiết từ authenticated frame và patch sign offsets. Output gồm bytes đã serialize và metrics/complete state. Caller chuyển segment output xuống stream tiếp theo. Nếu capacity không đủ, completion vẫn false và finalization phải báo lỗi thay vì giả thành công.

Listing 11: C++ source, dòng 1894–1939

```

1894 void AuthenticatedCavlcStreamDecoder::consume_segment(
1895     const std::vector<std::uint8_t>& annex_b_segment) {
1896     if (complete()) return;
1897     if (failed_) throw std::invalid_argument(failure_reason_);
1898     try {
1899         auto prepared = prepare_cavlc_stream_segment(annex_b_segment, parameter_sets_);
1900         parameter_sets_ = std::move(prepared.parameter_sets_);
1901         const auto& candidates = prepared.candidates;
1902         const auto segment_bit_count = std::min(maximum_bits_per_segment_, candidates.size());
1903         if (segment_bit_count == 0U) return;
1904         const auto segment_bits = extract_keyed_bits_from_candidates(
1905             prepared.analysis_input, secret_key_, candidates, segment_bit_count);
1906         for (const auto bit : segment_bits) {
1907             if (expected_frame_bits_.has_value() && collected_bits_.size() >= *expected_frame_bits_) break;
1908             collected_bits_.push_back(bit);
1909             if (!expected_frame_bits_.has_value() && collected_bits_.size() == 24U) {
1910                 const auto header = msb_bits_to_bytes(collected_bits_);
1911                 if (header[0] != 1U) throw std::invalid_argument("authenticated CAVLC stream version is invalid");
1912                 const auto payload_size = (static_cast<std::size_t>(header[1]) << 8U) | header[2];
1913                 if (payload_size > maximum_payload_bytes_) {
1914                     throw std::invalid_argument("authenticated CAVLC stream length exceeds configured maximum");
1915                 }
1916                 expected_frame_bits_ = (3U + payload_size + 16U) * 8U;
1917             }
1918         }
1919     }

```

```

1918         if (expected_frame_bits_.has_value() && collected_bits_.size() == *expected_frame_bits_) {
1919             payload_ = unpack_authenticated_cavlc_frame(
1920                 msb_bits_to_bytes(collected_bits_), secret_key_, maximum_payload_bytes_);
1921             break;
1922         }
1923     }
1924 } catch (const std::exception& error) {
1925     failed_ = true;
1926     failure_reason_ = error.what();
1927     wipe_stream_secret(collected_bits_);
1928     collected_bits_.clear();
1929     expected_frame_bits_.reset();
1930     parameter_sets_.clear();
1931     throw;
1932 }
1933 }
1934
1935 const std::vector<std::uint8_t>& AuthenticatedCavlcStreamDecoder::authenticated_payload() const {
1936     if (!payload_.has_value()) throw std::logic_error("authenticated CAVLC stream is incomplete");
1937     return *payload_;
1938 }

```

Nhận xét review. `AuthenticatedCavlcStreamDecoder::consume_segment()` làm phép đối xứng: parse context, tái tạo candidate order, append sign bits vào frame buffer, kiểm header/length/tag khi đã đủ dữ liệu và chuyển sang fail state khi thấy sai format. Chỉ `authenticated_payload()` sau complete mới được coi là payload đã qua frame authentication. Đây là extraction state machine, không phải Groth16 verifier.

6 Native CLI: process boundary và streaming I/O

6.1 Live embed

Listing 12: C++ source, dòng 367–448

```

367 void run_live_authenticated_embed(
368     std::istream& input,
369     std::ostream& output,
370     const std::vector<std::uint8_t>& key,
371     const std::vector<std::uint8_t>& payload,
372     const std::size_t maximum_bits_per_segment) {
373     zkstego::AuthenticatedCavlcStreamEncoder encoder(key, payload, maximum_bits_per_segment);
374     zkstego::AnnexBNalStreamReader reader(input);
375     std::vector<zkstego::AnnexBNalUnit> parameter_sets;
376     std::vector<std::uint8_t> nal_bytes;
377     std::vector<double> segment_latency_ms;
378     std::vector<double> idr_service_latency_ms;
379     segment_latency_ms.reserve(4096U);
380     idr_service_latency_ms.reserve(4096U);
381     std::size_t idr_count = 0;
382     std::size_t idr_segment_count = 0;
383     std::size_t candidate_capacity_bits = 0;
384     std::size_t bits_embedded = 0;
385     while (reader.read_next(nal_bytes)) {
386         const auto idr_service_started = std::chrono::steady_clock::now();
387         auto unit = parse_single_stream_nal(nal_bytes);
388         if (unit.nal_unit_type == 7U || unit.nal_unit_type == 8U) {
389             const auto existing = std::find_if(parameter_sets.begin(), parameter_sets.end(),
390                 [&](const auto& item) { return item.nal_unit_type == unit.nal_unit_type; });
391             if (existing == parameter_sets.end()) parameter_sets.push_back(unit);
392             else *existing = unit;
393             write_stream_bytes(output, nal_bytes);
394             continue;
395         }
396         if (unit.nal_unit_type != 5U) {
397             write_stream_bytes(output, nal_bytes);
398             continue;
399         }
400         add_capacity_count(idr_segment_count, 1U);
401         if (encoder.complete()) {
402             write_stream_bytes(output, nal_bytes);
403             if (idr_service_latency_ms.size() < 4096U) {
404                 idr_service_latency_ms.push_back(std::chrono::duration<double, std::milli>(
405                     std::chrono::steady_clock::now() - idr_service_started).count());
406             }
407             continue;
408         }
409         ++idr_count;
410         auto segment = parameter_sets;
411         segment.push_back(unit);
412         const auto processing_started = std::chrono::steady_clock::now();
413         const auto result = encoder.process_segment(zkstego::assemble_annex_b(segment));
414         candidate_capacity_bits += result.candidate_capacity;

```

```

415     bits_embedded += result.bits_embedded;
416     const auto processing_elapsed = std::chrono::duration<double, std::milli>(
417         std::chrono::steady_clock::now() - processing_started).count();
418     if (segment_latency_ms.size() < 4096U) segment_latency_ms.push_back(processing_elapsed);
419     const auto patched_units = zkstego::split_annex_b(result.output);
420     const auto patched_idr = std::find_if(patched_units.begin(), patched_units.end(),
421         [](const auto& candidate) { return candidate.nal_unit_type == 5U; });
422     if (patched_idr == patched_units.end() || std::find_if(std::next(patched_idr), patched_units.end(),
423         [](const auto& candidate) { return candidate.nal_unit_type == 5U; }) != patched_units.end()) {
424         throw std::invalid_argument("native stream patch did not return exactly one IDR slice");
425     }
426     write_stream_bytes(output, zkstego::assemble_annex_b(*patched_idr));
427     if (idr_service_latency_ms.size() < 4096U) {
428         idr_service_latency_ms.push_back(std::chrono::duration<double, std::milli>(
429             std::chrono::steady_clock::now() - idr_service_started).count());
430     }
431 }
432 if (idr_count == 0U) throw std::invalid_argument("input stream contained no IDR slices");
433 if (!encoder.complete()) throw std::invalid_argument("IDR stream ended before authenticated payload capacity was met");
434 std::cerr << "ZKSTEG_STREAM_METRICS {" << "patched_idr_segments\":" << idr_count
435     << ", \"idr_segment_count\":" << idr_segment_count
436     << ", \"idr_service_samples\":" << idr_service_latency_ms.size()
437     << ", \"idr_service_sample_limit\":" << 4096
438     << ", \"idr_service_p50_ms\":" << sample_quantile_ms(idr_service_latency_ms, 50U, 100U)
439     << ", \"idr_service_p95_ms\":" << sample_quantile_ms(idr_service_latency_ms, 95U, 100U)
440     << ", \"candidate_capacity_bits\":" << candidate_capacity_bits
441     << ", \"bits_embedded\":" << bits_embedded
442     << ", \"segment_process_samples\":" << segment_latency_ms.size()
443     << ", \"segment_process_sample_limit\":" << 4096
444     << ", \"segment_process_p50_ms\":" << sample_quantile_ms(segment_latency_ms, 50U, 100U)
445     << ", \"segment_process_p95_ms\":" << sample_quantile_ms(segment_latency_ms, 95U, 100U)
446     << "}" << "\n";
447 }

```

Nhận xét review. Hàm đọc key và payload control lines trước, sau đó dùng `AnnexBNalStreamReader` đọc media chunks/NAL incremental. Mỗi IDR segment được xử lý bởi authenticated stream encoder, bytes đầu ra ghi sang stdout. Dòng stderr/metrics phục vụ vận hành, không trộn vào media stdout. ‘stdin/stdout’ ở process boundary giúp API/WebSocket feed video mà không cần tạo file trung gian cho live route.

6.2 Live blind extraction

Listing 13: C++ source, dòng 449–477

```

449 std::vector<std::uint8_t> run_live_authenticated_extract(
450     std::istream& input,
451     const std::vector<std::uint8_t>& key,
452     const std::size_t maximum_payload_bytes,
453     const std::size_t maximum_bits_per_segment) {
454     zkstego::AuthenticatedCavlcStreamDecoder decoder(key, maximum_payload_bytes, maximum_bits_per_segment);
455     zkstego::AnnexBNalStreamReader reader(input);
456     std::vector<zkstego::AnnexBNalUnit> parameter_sets;
457     std::vector<std::uint8_t> nal_bytes;
458     std::size_t idr_count = 0;
459     while (reader.read_next(nal_bytes)) {
460         const auto unit = parse_single_stream_nal(nal_bytes);
461         if (unit.nal_unit_type == 7U || unit.nal_unit_type == 8U) {
462             const auto existing = std::find_if(parameter_sets.begin(), parameter_sets.end(),
463                 [&](const auto& item) { return item.nal_unit_type == unit.nal_unit_type; });
464             if (existing == parameter_sets.end()) parameter_sets.push_back(unit);
465             else *existing = unit;
466         } else if (unit.nal_unit_type == 5U) {
467             ++idr_count;
468             auto segment = parameter_sets;
469             segment.push_back(unit);
470             decoder.consume_segment(zkstego::assemble_annex_b(segment));
471             if (decoder.complete()) return decoder.authenticated_payload();
472         }
473     }
474     if (idr_count == 0U) throw std::invalid_argument("input stream contained no IDR slices");
475     throw std::invalid_argument("input stream ended before authenticated payload was complete");
476 }

```

Nhận xét review. Extractor đọc Annex-B từng NAL, cập nhật SPS/PPS, đưa từng IDR segment cho stateful decoder và dừng khi authenticated payload complete. Nó không nhận cover video gốc. Phần duy nhất cần ngoài stego stream là khóa và maximum payload bound. Output là raw payload; Python/application downstream mới unpack message/proof và gọi Groth16.

7 HTTP/WebSocket adapter và service API

7.1 Native adapter đầy đủ

Listing 14: Python source, dòng 29–494

```

29 def create_native_app(
30     settings: ApiSettings | None = None,
31     *,
32     cli_path: str | Path | None = None,
33     timeout_seconds: float = 120.0,
34     stream_timeout_seconds: float = 3600.0,
35 ) -> FastAPI:
36     """Create bounded HTTP-job and WebSocket-stream services backed by native authenticated CAVLC."""
37     if not math.isfinite(timeout_seconds) or timeout_seconds <= 0:
38         raise ValueError("timeout_seconds must be finite and positive")
39     if not math.isfinite(stream_timeout_seconds) or stream_timeout_seconds <= 0:
40         raise ValueError("stream_timeout_seconds must be finite and positive")
41     settings = settings or ApiSettings.from_environment()
42     configured_cli = cli_path or os.environ.get("ZK_STEGO_NATIVE_CLI")
43     if not configured_cli:
44         raise RuntimeError("ZK_STEGO_NATIVE_CLI must point to zkstego_blind_bits")
45     executable = Path(configured_cli).resolve(strict=True)
46     if not executable.is_file():
47         raise RuntimeError("ZK_STEGO_NATIVE_CLI must point to a regular executable file")
48     try:
49         maximum_bits_per_idr = int(os.environ.get("ZK_STEGO_MAX_BITS_PER_IDR", "64"))
50     except ValueError as exc:
51         raise ValueError("ZK_STEGO_MAX_BITS_PER_IDR must be a positive integer") from exc
52     if maximum_bits_per_idr <= 0:
53         raise ValueError("ZK_STEGO_MAX_BITS_PER_IDR must be a positive integer")
54
55     def invoke(
56         arguments: list[str], secret_key: bytes, *, stdin_tail: bytes = b"",
57     ) -> subprocess.CompletedProcess[bytes]:
58         if len(secret_key) != 32:
59             raise ValueError("native CAVLC key must be 32 bytes")
60         try:
61             result = subprocess.run(
62                 [str(executable), *arguments],
63                 input=secret_key.hex().encode("ascii") + b"\n" + stdin_tail,
64                 capture_output=True,
65                 timeout=timeout_seconds,
66                 check=False,
67             )
68         except subprocess.TimeoutExpired as exc:
69             raise RuntimeError("native CAVLC operation timed out") from exc
70         if result.returncode != 0:
71             raise RuntimeError("native CAVLC operation failed")
72         return result
73
74     def embed_handler(
75         *, video_path: str, message: bytes, output_path: str, secret_key: bytes, **_: Any,
76     ) -> dict[str, Any]:
77         invoke(
78             ["embed-stream-auth-stdin", video_path, output_path, str(maximum_bits_per_idr)],
79             secret_key,
80             stdin_tail=message.hex().encode("ascii") + b"\n",
81         )
82         if not Path(output_path).is_file():
83             raise RuntimeError("native CAVLC operation did not create output")
84         # IDR segments carry one authenticated frame: version + length + HMAC tag + payload.
85         return {"valid": True, "bits_embedded": (len(message) + 19) * 8, "output_file": Path(output_path).name}
86
87     def extract_handler(
88         *, stego_video_path: str, output_path: str, secret_key: bytes, maximum_payload_bytes: int,
89     ) -> dict[str, Any]:
90         result = invoke(
91             ["extract-stream-auth", stego_video_path, "-", str(maximum_payload_bytes), str(maximum_bits_per_idr)],
92             secret_key,
93         )
94         try:
95             payload = bytes.fromhex(result.stdout.decode("ascii").strip())
96         except (UnicodeDecodeError, ValueError) as exc:
97             raise RuntimeError("native CAVLC extractor returned malformed payload") from exc
98         if len(payload) > maximum_payload_bytes:
99             raise RuntimeError("native CAVLC extractor exceeded requested payload bound")
100         destination = Path(output_path)
101         try:
102             with destination.open("wb") as stream:
103                 stream.write(payload)
104         except BaseException:
105             destination.unlink(missing_ok=True)
106             raise
107         return {"valid": True, "payload_bytes": len(payload)}
108
109     app = create_app(
110         settings,
111         embed_handler=embed_handler,

```

```

112     verify_handler=None,
113     extract_handler=extract_handler,
114 )
115
116 @app.websocket("/api/v1/stream")
117 async def native_stream(websocket: WebSocket) -> None:
118     authorization = websocket.headers.get("authorization")
119     expected = f"Bearer {settings.api_token}"
120     await websocket.accept()
121     if not authorization or not secrets.compare_digest(authorization, expected):
122         await websocket.close(code=4401, reason="authentication required")
123     return
124     if not app.state.capacity.acquire(blocking=False):
125         await websocket.send_json({"type": "error", "code": "stream_capacity_reached"})
126         await websocket.close(code=1013, reason="stream capacity reached")
127     return
128
129 process: asyncio.subprocess.Process | None = None
130 input_task: asyncio.Task[None] | None = None
131 output_task: asyncio.Task[None] | None = None
132 resource_task: asyncio.Task[None] | None = None
133 native_resources: dict[str, int | float] = {
134     "sample_count": 0,
135     "peak_rss_bytes": 0,
136     "cpu_seconds": 0.0,
137     "sample_interval_ms": 100,
138 }
139 accepted = True
140 try:
141     try:
142         start_event = await asyncio.wait_for(websocket.receive(), timeout=10.0)
143         start_text = start_event.get("text")
144         if start_event.get("type") != "websocket.receive" or start_text is None or len(start_text) > 12 * 1024:
145             raise ValueError("invalid stream control message size")
146         start = json.loads(start_text)
147         if not isinstance(start, dict) or start.get("operation") not in {"embed", "extract"}:
148             raise ValueError("invalid stream operation")
149         operation = start["operation"]
150         expected_fields = {
151             "operation", "secret_key_b64", "message_b64"
152             if operation == "embed"
153             else {"operation", "secret_key_b64", "maximum_payload_bytes"}
154         }
155         if set(start) != expected_fields:
156             raise ValueError("invalid stream control message")
157         key_text = start["secret_key_b64"]
158         if not isinstance(key_text, str):
159             raise ValueError("invalid key encoding")
160         key = base64.b64decode(key_text, validate=True)
161         if len(key) != 32:
162             raise ValueError("invalid key length")
163         if operation == "embed":
164             message_text = start["message_b64"]
165             if not isinstance(message_text, str):
166                 raise ValueError("invalid payload encoding")
167             message = base64.b64decode(message_text, validate=True)
168             if not message or len(message) > 4096:
169                 raise ValueError("invalid payload length")
170             arguments = ["embed-live-auth-stdin", str(maximum_bits_per_idr)]
171             preamble = key.hex().encode("ascii") + b"\n" + message.hex().encode("ascii") + b"\n"
172             maximum_payload_bytes = 0
173         else:
174             maximum_payload_bytes = start["maximum_payload_bytes"]
175             if (not isinstance(maximum_payload_bytes, int) or isinstance(maximum_payload_bytes, bool)
176                 or not 1 <= maximum_payload_bytes <= 4096):
177                 raise ValueError("invalid payload bound")
178             arguments = [
179                 "extract-live-auth-stdin", str(maximum_payload_bytes), str(maximum_bits_per_idr),
180             ]
181             preamble = key.hex().encode("ascii") + b"\n"
182         del key
183     except (asyncio.TimeoutError, KeyError, TypeError, ValueError, UnicodeError):
184         await websocket.send_json({"type": "error", "code": "invalid_stream_request"})
185         await websocket.close(code=4400, reason="invalid stream request")
186     return
187
188     process = await asyncio.create_subprocess_exec(
189         str(executable), *arguments,
190         stdin=asyncio.subprocess.PIPE,
191         stdout=asyncio.subprocess.PIPE,
192         stderr=asyncio.subprocess.PIPE,
193         limit=64 * 1024,
194     )
195     assert process.stdin is not None and process.stdout is not None and process.stderr is not None
196     native_stdin_high_water_bytes = 64 * 1024
197     native_stdin_low_water_bytes = 16 * 1024
198     stdin_transport = process.stdin.transport
199     set_write_limits = getattr(stdin_transport, "set_write_buffer_limits", None)
200     get_write_limits = getattr(stdin_transport, "get_write_buffer_limits", None)
201     get_write_buffer_size = getattr(stdin_transport, "get_write_buffer_size", None)
202     if not callable(set_write_limits) or not callable(get_write_limits) or not callable(get_write_buffer_size):

```

```

203     raise RuntimeError("native stdin transport does not expose bounded flow control")
204 set_write_limits(high=native_stdin_high_water_bytes, low=native_stdin_low_water_bytes)
205 _stdin_low, observed_stdin_high = get_write_limits()
206 if observed_stdin_high != native_stdin_high_water_bytes:
207     raise RuntimeError("native stdin flow-control high-water mark was not applied")
208 async def sample_native_resources() -> None:
209     assert process is not None
210     started_at = time.perf_counter()
211     try:
212         monitored = psutil.Process(process.pid)
213     except (psutil.NoSuchProcess, psutil.AccessDenied):
214         return
215     while process.returncode is None:
216         try:
217             native_resources["peak_rss_bytes"] = max(
218                 int(native_resources["peak_rss_bytes"]), monitored.memory_info().rss,
219             )
220             cpu = monitored.cpu_times()
221             native_resources["cpu_seconds"] = cpu.user + cpu.system
222             native_resources["sample_count"] = int(native_resources["sample_count"]) + 1
223         except (psutil.NoSuchProcess, psutil.AccessDenied):
224             break
225         await asyncio.sleep(0.1)
226     native_resources["wall_seconds"] = round(time.perf_counter() - started_at, 6)
227
228 resource_task = asyncio.create_task(sample_native_resources())
229 process.stdin.write(preamble)
230 await asyncio.wait_for(process.stdin.drain(), timeout=10.0)
231 deadline = asyncio.get_running_loop().time() + stream_timeout_seconds
232 media_bytes_received = 0
233 maximum_stream_bytes = settings.max_upload_bytes
234 maximum_ws_chunk_bytes = min(1024 * 1024, settings.max_upload_bytes)
235 maximum_ws_chunks = 100_000
236 chunks_received = 0
237 flow_control: dict[str, int | float | str] = {
238     "buffering_model": "one-websocket-message-at-a-time-with-await-drain",
239     "input_chunks": 0,
240     "input_bytes": 0,
241     "input_chunk_limit_bytes": maximum_ws_chunk_bytes,
242     "max_input_chunk_bytes": 0,
243     "native_stdin_low_water_bytes": native_stdin_low_water_bytes,
244     "native_stdin_high_water_bytes": native_stdin_high_water_bytes,
245     "max_native_stdin_buffer_bytes": 0,
246     "input_drain_count": 0,
247     "input_drain_wait_ms_total": 0.0,
248     "output_chunks": 0,
249     "output_chunk_limit_bytes": 64 * 1024,
250     "max_output_chunk_bytes": 0,
251     "dropped_input_chunks": 0,
252 }
253 input_work_samples: deque[float] = deque(maxlen=4096)
254 output_send_samples: deque[float] = deque(maxlen=4096)
255 output_read_wait_samples: deque[float] = deque(maxlen=4096)
256
257 def summarize_samples(samples: deque[float], prefix: str) -> None:
258     ordered = sorted(samples)
259
260     def percentile(percent: int) -> float:
261         rank = math.ceil(percent * len(ordered) / 100)
262         return round(ordered[max(rank - 1, 0)], 4)
263
264     flow_control[f"{prefix}_samples"] = len(ordered)
265     flow_control[f"{prefix}_p50_ms"] = percentile(50)
266     flow_control[f"{prefix}_p95_ms"] = percentile(95)
267
268 await websocket.send_json({"type": "ready", "operation": operation})
269
270 async def feed_native_stdin() -> None:
271     nonlocal chunks_received, media_bytes_received
272     assert process is not None and process.stdin is not None
273     while True:
274         remaining = deadline - asyncio.get_running_loop().time()
275         if remaining <= 0:
276             raise TimeoutError("native stream deadline exceeded")
277         event = await asyncio.wait_for(websocket.receive(), timeout=remaining)
278         if event["type"] == "websocket.disconnect":
279             process.stdin.close()
280             return
281         data = event.get("bytes")
282         if data is not None:
283             chunks_received += 1
284             if (not data or len(data) > maximum_ws_chunk_bytes
285                 or chunks_received > maximum_ws_chunks):
286                 raise ValueError("invalid stream chunk size")
287             media_bytes_received += len(data)
288             if media_bytes_received > maximum_stream_bytes:
289                 raise ValueError("stream byte limit exceeded")
290             flow_control["input_chunks"] = int(flow_control["input_chunks"]) + 1
291             flow_control["input_bytes"] = media_bytes_received
292             flow_control["max_input_chunk_bytes"] = max(
293                 int(flow_control["max_input_chunk_bytes"]), len(data),

```

```

294         )
295         process.stdin.write(data)
296         flow_control["max_native_stdin_buffer_bytes"] = max(
297             int(flow_control["max_native_stdin_buffer_bytes"]), get_write_buffer_size(),
298         )
299         try:
300             remaining = deadline - asyncio.get_running_loop().time()
301             if remaining <= 0:
302                 raise TimeoutError("native stream deadline exceeded")
303             write_drain_started = time.perf_counter()
304             await asyncio.wait_for(process.stdin.drain(), timeout=remaining)
305             input_work_samples.append((time.perf_counter() - write_drain_started) * 1000.0)
306             flow_control["input_drain_count"] = int(flow_control["input_drain_count"]) + 1
307             flow_control["input_drain_wait_ms_total"] = (
308                 float(flow_control["input_drain_wait_ms_total"])
309                 + (time.perf_counter() - write_drain_started) * 1000.0
310             )
311             flow_control["max_native_stdin_buffer_bytes"] = max(
312                 int(flow_control["max_native_stdin_buffer_bytes"]), get_write_buffer_size(),
313             )
314         except (BrokenPipeError, ConnectionResetError):
315             if operation == "extract":
316                 return
317             raise
318         continue
319     text = event.get("text")
320     if text is None or len(text) > 128:
321         raise ValueError("invalid stream control frame")
322     control = json.loads(text)
323     if control != {"type": "end"}:
324         raise ValueError("invalid stream control frame")
325     process.stdin.close()
326     with contextlib.suppress(BrokenPipeError, ConnectionResetError):
327         await process.stdin.wait_closed()
328     return
329
330 async def forward_native_output() -> None:
331     assert process is not None and process.stdout is not None
332     if operation == "embed":
333         while True:
334             remaining = deadline - asyncio.get_running_loop().time()
335             if remaining <= 0:
336                 raise TimeoutError("native stream deadline exceeded")
337             read_started = time.perf_counter()
338             chunk = await asyncio.wait_for(process.stdout.read(64 * 1024), timeout=remaining)
339             output_read_wait_samples.append((time.perf_counter() - read_started) * 1000.0)
340             if not chunk:
341                 break
342             flow_control["output_chunks"] = int(flow_control["output_chunks"]) + 1
343             flow_control["max_output_chunk_bytes"] = max(
344                 int(flow_control["max_output_chunk_bytes"]), len(chunk),
345             )
346             send_started = time.perf_counter()
347             await websocket.send_bytes(chunk)
348             output_send_samples.append((time.perf_counter() - send_started) * 1000.0)
349             remaining = deadline - asyncio.get_running_loop().time()
350             if remaining <= 0:
351                 raise TimeoutError("native stream deadline exceeded")
352             return_code = await asyncio.wait_for(process.wait(), timeout=remaining)
353             if return_code == 0:
354                 if resource_task is not None:
355                     with contextlib.suppress(asyncio.TimeoutError):
356                         await asyncio.wait_for(asyncio.shield(resource_task), timeout=0.5)
357                 assert process.stderr is not None
358                 native_diagnostics = await process.stderr.read()
359                 stream_metrics = None
360                 for line in native_diagnostics.decode("utf-8", errors="replace").splitlines():
361                     if line.startswith("ZKSTEG_STREAM_METRICS "):
362                         try:
363                             candidate_metrics = json.loads(line.removeprefix("ZKSTEG_STREAM_METRICS "))
364                         except (TypeError, ValueError):
365                             continue
366                         if isinstance(candidate_metrics, dict)
367                             and set(candidate_metrics) == {
368                                 "patched_idr_segments", "segment_process_samples",
369                                 "segment_process_sample_limit",
370                                 "segment_process_p50_ms", "segment_process_p95_ms",
371                                 "candidate_capacity_bits", "bits_embedded",
372                                 "idr_segment_count", "idr_service_samples",
373                                 "idr_service_sample_limit", "idr_service_p50_ms",
374                                 "idr_service_p95_ms",
375                             }
376                             and type(candidate_metrics["patched_idr_segments"]) is int
377                             and candidate_metrics["patched_idr_segments"] > 0
378                             and type(candidate_metrics["idr_segment_count"]) is int
379                             and candidate_metrics["idr_segment_count"] >= candidate_metrics["patched_idr_segments"]
380                             and type(candidate_metrics["idr_service_samples"]) is int
381                             and type(candidate_metrics["idr_service_sample_limit"]) is int
382                             and candidate_metrics["idr_service_sample_limit"] == 4096
383                             and candidate_metrics["idr_service_samples"] == min(
384                                 candidate_metrics["idr_segment_count"],

```

```

385         candidate_metrics["idr_service_sample_limit"],
386     )
387     and type(candidate_metrics["candidate_capacity_bits"]) is int
388     and type(candidate_metrics["bits_embedded"]) is int
389     and candidate_metrics["bits_embedded"] > 0
390     and candidate_metrics["candidate_capacity_bits"] >= candidate_metrics["bits_embedded"]
391     and type(candidate_metrics["segment_process_samples"]) is int
392     and 1 <= candidate_metrics["segment_process_samples"] <= 4096
393     and candidate_metrics["segment_process_samples"] <= candidate_metrics["
patched_idr_segments"]
394
395     and candidate_metrics["segment_process_sample_limit"] == 4096
396     and all(
397         isinstance(candidate_metrics[field], (int, float))
398         and not isinstance(candidate_metrics[field], bool)
399         and math.isfinite(candidate_metrics[field])
400         and candidate_metrics[field] >= 0
401         for field in (
402             "segment_process_p50_ms", "segment_process_p95_ms",
403             "idr_service_p50_ms", "idr_service_p95_ms",
404         ))):
405     if (candidate_metrics["segment_process_p50_ms"] > candidate_metrics["segment_process_p95_ms"]
406         or candidate_metrics["idr_service_p50_ms"] > candidate_metrics["idr_service_p95_ms"]
407     ):
408         continue
409     stream_metrics = candidate_metrics
410     break
411     response = {"type": "complete", "media_bytes": media_bytes_received}
412     if input_work_samples:
413         summarize_samples(input_work_samples, "native_stdin_write_drain")
414     if output_send_samples:
415         summarize_samples(output_send_samples, "native_stdout_to_websocket_send")
416     if output_read_wait_samples:
417         summarize_samples(output_read_wait_samples, "native_stdout_read_wait")
418     response["flow_control"] = flow_control
419     if stream_metrics is not None:
420         response["native_metrics"] = stream_metrics
421     if native_resources["sample_count"]:
422         response["native_resources"] = native_resources
423     await websocket.send_json(response)
424 else:
425     await websocket.send_json({"type": "error", "code": "incomplete_or_invalid_stream"})
426 return
427
428 remaining = deadline - asyncio.get_running_loop().time()
429 if remaining <= 0:
430     raise TimeoutError("native stream deadline exceeded")
431 output = await asyncio.wait_for(process.stdout.readline(), timeout=remaining)
432 if len(output) > maximum_payload_bytes * 2 + 1:
433     await websocket.send_json({"type": "error", "code": "invalid_native_response"})
434     return
435 remaining = deadline - asyncio.get_running_loop().time()
436 if remaining <= 0:
437     raise TimeoutError("native stream deadline exceeded")
438 return_code = await asyncio.wait_for(process.wait(), timeout=remaining)
439 if return_code != 0:
440     await websocket.send_json({"type": "error", "code": "payload_not_authenticated"})
441     return
442 try:
443     payload = bytes.fromhex(output.decode("ascii").strip())
444 except (UnicodeDecodeError, ValueError):
445     await websocket.send_json({"type": "error", "code": "invalid_native_response"})
446     return
447 if not payload or len(payload) > maximum_payload_bytes:
448     await websocket.send_json({"type": "error", "code": "invalid_native_response"})
449     return
450 await websocket.send_json({
451     "type": "payload", "payload_b64": base64.b64encode(payload).decode("ascii"),
452 })
453
454 input_task = asyncio.create_task(feed_native_stdin())
455 output_task = asyncio.create_task(forward_native_output())
456 done, _ = await asyncio.wait((input_task, output_task), return_when=asyncio.FIRST_COMPLETED)
457 if output_task in done:
458     await output_task
459     if not input_task.done():
460         input_task.cancel()
461         with contextlib.suppress(asyncio.CancelledError):
462             await input_task
463 else:
464     await input_task
465     await output_task
466 except (WebSocketDisconnect, ConnectionError, BrokenPipeError):
467     pass
468 except Exception: # noqa: BLE001 - errors are deliberately generic on the wire.
469     if accepted:
470         with contextlib.suppress(RuntimeError, WebSocketDisconnect):
471             await websocket.send_json({"type": "error", "code": "stream_failed"})
472             await websocket.close(code=1011, reason="stream failed")
473 finally:

```

```

473     try:
474         for task in (input_task, output_task):
475             if task is not None and not task.done():
476                 task.cancel()
477                 with contextlib.suppress(asyncio.CancelledError, Exception):
478                     await task
479             if resource_task is not None and not resource_task.done():
480                 resource_task.cancel()
481                 with contextlib.suppress(asyncio.CancelledError):
482                     await resource_task
483         if process is not None:
484             if process.stdin is not None and not process.stdin.is_closing():
485                 process.stdin.close()
486             if process.returncode is None:
487                 with contextlib.suppress(ProcessLookupError):
488                     process.kill()
489                 with contextlib.suppress(ProcessLookupError):
490                     await process.wait()
491         finally:
492             app.state.capacity.release()
493
494     return app

```

Nhận xét review. Đây là adapter native đầy đủ: `invoke()` cấp key cho child qua `stdin` thay vì `argv/temp` key file; `embed/extract` handlers gọi CLI authenticated; `create_app(... verify_handler=None)` chủ động không giả vờ rằng native process verify Groth16; WebSocket nhận control JSON trước binary chunks rồi forward qua `stdin/stdout` với bounds, timeout, backpressure và metrics. Review cần tập trung vào lifecycle child process, hủy task khi socket disconnect, chunk/total byte limits, semaphore cleanup và đảm bảo key không được đưa vào log. API token chỉ bảo vệ dịch vụ HTTP; nó không phải secret key CAVLC/HMAC.

7.2 HTTP job service

Listing 15: Python source, dòng 315–669

```

315 def create_app(
316     settings: ApiSettings | None = None,
317     *,
318     embed_handler: Callable[..., Any] = embed,
319     verify_handler: Callable[..., Any] | None = verify,
320     extract_handler: Callable[..., Any] | None = None,
321 ) -> FastAPI:
322     settings = settings or ApiSettings.from_environment()
323     if settings.max_workers < 1 or settings.max_queued_jobs < 0:
324         raise ValueError("max_workers must be positive and max_queued_jobs must be non-negative")
325     if not math.isfinite(settings.artifact_ttl_seconds) or settings.artifact_ttl_seconds < 1.0:
326         raise ValueError("artifact_ttl_seconds must be finite and at least one second")
327     if not math.isfinite(settings.upload_timeout_seconds) or settings.upload_timeout_seconds < 1.0:
328         raise ValueError("upload_timeout_seconds must be finite and at least one second")
329     app = FastAPI(title="ZK-Stego Video API", version="1.0.0", docs_url="/docs", redoc_url=None)
330     app.add_middleware(
331         RequestBodyGuard,
332         maximum_bytes={
333             "/api/v1/jobs/embed": settings.max_upload_bytes + (1024 * 1024),
334             "/api/v1/jobs/extract": settings.max_upload_bytes + (1024 * 1024),
335             "/api/v1/jobs/verify": (2 * settings.max_upload_bytes) + (1024 * 1024),
336         },
337         timeout_seconds=settings.upload_timeout_seconds,
338     )
339     store = JobStore(settings.work_dir / "jobs")
340     work_dir_lease = WorkDirLease(settings.work_dir / ".api-instance.lock")
341     executor = ThreadPoolExecutor(max_workers=settings.max_workers, thread_name_prefix="zk-stego")
342     capacity = threading.BoundedSemaphore(settings.max_workers + settings.max_queued_jobs)
343     artifact_cleanup_stop = threading.Event()
344     artifact_cleanup_thread: threading.Thread | None = None
345     artifact_lock = threading.Lock()
346     app.state.settings, app.state.store, app.state.executor = settings, store, executor
347     app.state.capacity = capacity
348     app.state.work_dir_lease = work_dir_lease
349
350     def require_token(authorization: str | None = Header(default=None)) -> None:
351         expected = f"Bearer {settings.api_token}"
352         if not authorization or not secrets.compare_digest(authorization, expected):
353             raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Bearer token required")
354
355     def reserve_capacity() -> None:
356         if not capacity.acquire(blocking=False):
357             raise HTTPException(status_code=503, detail="job capacity reached; retry later", headers={"Retry-After": "1"})
358
359     def cleanup_inputs(job_id: str, operation: str) -> None:
360         directory = settings.work_dir / "files" / job_id
361         filenames = {
362             "embed": ("cover.h264"),

```

```

363         "verify": ("stego.h264", "original.h264"),
364         "extract": ("stego.h264",),
365     }.get(operation, ())
366     for filename in filenames:
367         try:
368             (directory / filename).unlink(missing_ok=True)
369         except OSError:
370             continue
371
372 def submit(job_id: str, operation: str, handler: Callable[[], dict[str, Any]]) -> None:
373     def run() -> None:
374         try:
375             store.update(job_id, status="running")
376             try:
377                 result = handler()
378                 store.update(job_id, status="succeeded", result=result)
379             except Exception as exc: # Do not expose paths, secrets, or internal tracebacks. # noqa: BLE001
380                 store.update(job_id, status="failed", error=f"{operation} failed: {type(exc).__name__}")
381             finally:
382                 try:
383                     cleanup_inputs(job_id, operation)
384                 finally:
385                     capacity.release()
386         try:
387             executor.submit(run)
388         except Exception:
389             try:
390                 store.update(job_id, status="failed", error=f"{operation} could not be scheduled")
391             finally:
392                 try:
393                     cleanup_inputs(job_id, operation)
394                 finally:
395                     capacity.release()
396         raise
397
398 def abandon_upload(job_id: str, job_dir: Path, created: bool) -> None:
399     if created:
400         for path in job_dir.iterdir():
401             if path.is_file():
402                 path.unlink(missing_ok=True)
403             job_dir.rmdir()
404         store.discard(job_id)
405
406 def sweep_expired_extract_artifacts() -> None:
407     cutoff = time.time() - settings.artifact_ttl_seconds
408     files_root = settings.work_dir / "files"
409     try:
410         job_dirs = tuple(files_root.iterdir())
411     except OSError:
412         return
413     with artifact_lock:
414         for job_dir in job_dirs:
415             if not job_dir.is_dir():
416                 continue
417             job_id = job_dir.name
418             record = store.get(job_id)
419             if not record:
420                 continue
421             if record.get("operation") == "embed":
422                 main_artifact = job_dir / "stego.h264"
423             elif record.get("operation") == "extract":
424                 main_artifact = job_dir / "payload.bin"
425             else:
426                 continue
427             artifacts = (main_artifact, *job_dir.glob(".artifact-claim-*.tmp"))
428             removed = False
429             for artifact_path in artifacts:
430                 try:
431                     if artifact_path.stat().st_mtime < cutoff:
432                         artifact_path.unlink(missing_ok=True)
433                         removed = True
434                 except OSError:
435                     continue
436             if removed:
437                 try:
438                     record = store.get(job_id)
439                     if record and record.get("operation") in {"embed", "extract"}:
440                         store.update(job_id, artifact_expired=True)
441                 except (OSError, KeyError):
442                     continue
443
444 def run_artifact_cleanup() -> None:
445     interval = min(60.0, max(1.0, settings.artifact_ttl_seconds / 2.0))
446     while not artifact_cleanup_stop.wait(interval):
447         sweep_expired_extract_artifacts()
448
449 def recover_interrupted_jobs() -> None:
450     files_root = settings.work_dir / "files"
451     for record in store.all_records():
452         job_id = record.get("job_id")
453         if not isinstance(job_id, str) or re.fullmatch(r"[A-Za-z0-9_-]{24}", job_id) is None:

```

```

454         continue
455     try:
456         if record.get("status") not in {"queued", "running"}:
457             continue
458         operation = record.get("operation")
459         if not isinstance(operation, str) or operation not in {"embed", "verify", "extract"}:
460             continue
461         cleanup_inputs(job_id, operation)
462         job_dir = files_root / job_id
463         partial_outputs = {
464             "embed": ("stego.h264",),
465             "extract": ("payload.bin",),
466         }.get(operation, ())
467         if job_dir.is_dir():
468             for filename in partial_outputs:
469                 (job_dir / filename).unlink(missing_ok=True)
470             store.update(job_id, status="failed", error="job interrupted by service restart")
471     except (OSError, KeyError):
472         continue
473
474 @app.on_event("startup")
475 def startup() -> None:
476     nonlocal artifact_cleanup_thread
477     work_dir_lease.acquire()
478     try:
479         recover_interrupted_jobs()
480         sweep_expired_extract_artifacts()
481         artifact_cleanup_stop.clear()
482         artifact_cleanup_thread = threading.Thread(
483             target=run_artifact_cleanup, name="zk-stego-artifact-cleanup", daemon=True,
484         )
485         artifact_cleanup_thread.start()
486         app.state.artifact_cleanup_thread = artifact_cleanup_thread
487         app.state.sweep_expired_extract_artifacts = sweep_expired_extract_artifacts
488         app.state.recover_interrupted_jobs = recover_interrupted_jobs
489     except BaseException:
490         work_dir_lease.release()
491         raise
492
493 @app.on_event("shutdown")
494 def shutdown() -> None:
495     artifact_cleanup_stop.set()
496     if artifact_cleanup_thread is not None:
497         artifact_cleanup_thread.join(timeout=2.0)
498     try:
499         executor.shutdown(wait=True, cancel_futures=False)
500     finally:
501         work_dir_lease.release()
502
503 @app.get("/health")
504 def health() -> dict[str, str]:
505     return {"status": "ok", "service": "zk-stego-video"}
506
507 @app.post("/api/v1/jobs/embed", status_code=status.HTTP_202_ACCEPTED, dependencies=[Depends(require_token)])
508 async def start_embed(
509     video: UploadFile = File(...), # noqa: B008
510     message_b64: str = Form(...),
511     secret_key_b64: str = Form(...),
512     chaos_key_b64: str | None = Form(default=None),
513     ffmpeg_validate: bool = Form(default=False),
514 ) -> dict[str, str]:
515     message, secret_key = _decode_message(message_b64), _decode_key(secret_key_b64, "secret_key_b64")
516     chaos_key = _decode_key(chaos_key_b64, "chaos_key_b64") if chaos_key_b64 else None
517     reserve_capacity()
518     job_dir_created = False
519     try:
520         record = store.create("embed")
521         job_dir = settings.work_dir / "files" / record["job_id"]
522         job_dir.mkdir(parents=True, exist_ok=False)
523         job_dir_created = True
524         input_path, output_path = job_dir / "cover.h264", job_dir / "stego.h264"
525         await _save_upload(video, input_path, settings.max_upload_bytes)
526     except BaseException:
527         try:
528             if "record" in locals():
529                 abandon_upload(record["job_id"], job_dir, job_dir_created)
530         finally:
531             capacity.release()
532         raise
533
534 def perform() -> dict[str, Any]:
535     result = embed_handler(
536         video_path=str(input_path), message=message, output_path=str(output_path),
537         circuits_dir=str(settings.circuits_dir), secret_key=secret_key, chaos_key=chaos_key,
538         ffmpeg_validate=ffmpeg_validate,
539     )
540     return _safe_embed_result(result, output_path.name)
541
542 submit(record["job_id"], "embed", perform)
543 return {"job_id": record["job_id"], "status": "queued"}
544

```

```

545 @app.post("/api/v1/jobs/verify", status_code=status.HTTP_202_ACCEPTED, dependencies=[Depends(require_token)])
546 async def start_verify(
547     stego_video: UploadFile = File(...), # noqa: B008
548     original_video: UploadFile = File(...), # noqa: B008
549     secret_key_b64: str = Form(...),
550     message_length: int = Form(..., ge=1, le=4096),
551     chaos_key_b64: str | None = Form(default=None),
552 ) -> dict[str, str]:
553     if verify_handler is None:
554         raise HTTPException(status_code=501, detail="proof verification is not configured for this service")
555     secret_key = _decode_key(secret_key_b64, "secret_key_b64")
556     chaos_key = _decode_key(chaos_key_b64, "chaos_key_b64") if chaos_key_b64 else None
557     reserve_capacity()
558     job_dir_created = False
559     try:
560         record = store.create("verify")
561         job_dir = settings.work_dir / "files" / record["job_id"]
562         job_dir.mkdir(parents=True, exist_ok=False)
563         job_dir_created = True
564         stego_path, original_path = job_dir / "stego.h264", job_dir / "original.h264"
565         await _save_upload(stego_video, stego_path, settings.max_upload_bytes)
566         await _save_upload(original_video, original_path, settings.max_upload_bytes)
567     except BaseException:
568         try:
569             if "record" in locals():
570                 abandon_upload(record["job_id"], job_dir, job_dir_created)
571         finally:
572             capacity.release()
573         raise
574
575     def perform() -> dict[str, Any]:
576         result = verify_handler(
577             stego_video_path=str(stego_path), original_video_path=str(original_path),
578             circuits_dir=str(settings.circuits_dir), secret_key=secret_key,
579             message_length=message_length, chaos_key=chaos_key,
580         )
581         return _safe_verify_result(result)
582
583     submit(record["job_id"], "verify", perform)
584     return {"job_id": record["job_id"], "status": "queued"}
585
586 @app.post("/api/v1/jobs/extract", status_code=status.HTTP_202_ACCEPTED, dependencies=[Depends(require_token)])
587 async def start_extract(
588     stego_video: UploadFile = File(...), # noqa: B008
589     secret_key_b64: str = Form(...),
590     maximum_payload_bytes: int = Form(..., ge=1, le=4096),
591 ) -> dict[str, str]:
592     if extract_handler is None:
593         raise HTTPException(status_code=501, detail="blind extraction is not configured for this service")
594     secret_key = _decode_key(secret_key_b64, "secret_key_b64")
595     reserve_capacity()
596     job_dir_created = False
597     try:
598         record = store.create("extract")
599         job_dir = settings.work_dir / "files" / record["job_id"]
600         job_dir.mkdir(parents=True, exist_ok=False)
601         job_dir_created = True
602         stego_path, payload_path = job_dir / "stego.h264", job_dir / "payload.bin"
603         await _save_upload(stego_video, stego_path, settings.max_upload_bytes)
604     except BaseException:
605         try:
606             if "record" in locals():
607                 abandon_upload(record["job_id"], job_dir, job_dir_created)
608         finally:
609             capacity.release()
610         raise
611
612     def perform() -> dict[str, Any]:
613         result = extract_handler(
614             stego_video_path=str(stego_path), output_path=str(payload_path),
615             secret_key=secret_key, maximum_payload_bytes=maximum_payload_bytes,
616         )
617         return _safe_extract_result(result, payload_path.name)
618
619     submit(record["job_id"], "extract", perform)
620     return {"job_id": record["job_id"], "status": "queued"}
621
622 @app.get("/api/v1/jobs/{job_id}", dependencies=[Depends(require_token)])
623 def get_job(job_id: str) -> dict[str, Any]:
624     record = store.get(job_id)
625     if record is None:
626         raise HTTPException(status_code=404, detail="job not found")
627     return record
628
629 @app.get("/api/v1/jobs/{job_id}/artifact", dependencies=[Depends(require_token)])
630 def download_artifact(job_id: str) -> FileResponse:
631     record = store.get(job_id)
632     if not record or record.get("status") != "succeeded":
633         raise HTTPException(status_code=404, detail="artifact not available")
634     if record.get("operation") == "embed":
635         source = settings.work_dir / "files" / job_id / "stego.h264"

```

```

636     media_type, filename = "video/h264", "stego.h264"
637     elif record.get("operation") == "extract":
638         source = settings.work_dir / "files" / job_id / "payload.bin"
639         media_type, filename = "application/octet-stream", "payload.bin"
640     else:
641         raise HTTPException(status_code=404, detail="artifact not available")
642     claimed = source.with_name(f".artifact-claim-{{secrets.token_hex(12)}}.tmp")
643     with artifact_lock:
644         try:
645             if source.stat().st_mtime < time.time() - settings.artifact_ttl_seconds:
646                 source.unlink(missing_ok=True)
647                 store.update(job_id, artifact_expired=True)
648                 raise HTTPException(status_code=404, detail="artifact not available")
649             os.replace(source, claimed)
650             os.utime(claimed, None)
651         except FileNotFoundError as exc:
652             raise HTTPException(status_code=404, detail="artifact not available") from exc
653
654     def consume_artifact() -> None:
655         try:
656             with artifact_lock:
657                 claimed.unlink(missing_ok=True)
658                 store.update(job_id, artifact_expired=True)
659         except (OSError, KeyError):
660             pass
661
662     if not claimed.is_file():
663         raise HTTPException(status_code=404, detail="artifact not available")
664     return FileResponse(
665         claimed, media_type=media_type, filename=filename,
666         background=BackgroundTask(consume_artifact),
667     )
668
669     return app

```

Nhận xét review. `create_app()` ghép các dependency đã định nghĩa ở đầu file: body guard, auth token, workdir lease, JobStore, semaphore, worker, cleanup/recovery và route embed/verify/extract/artifact. Đây là protocol/job orchestration layer. '202 Accepted' nghĩa là job đã được đưa vào queue; client phải poll job rồi tải artifact. Kết quả HTTP không tự có nghĩa là video decode tốt hay Groth16 proof hợp lệ. Với native adapter đang dùng, verify handler là None nên route verify trả 501 thay vì đưa ra kết luận giả.

8 Python full-video route: cùng hệ thống, implementation khác

8.1 Orchestrator nhúng của Python

Listing 16: Python source, dòng 210–574

```

210 -
211 def embed(
212     video_path: str,
213     message: bytes,
214     output_path: str,
215     circuits_dir: str,
216     secret_key: bytes,
217     max_modifications_per_block: int = 1,
218     ffmpeg_validate: bool = False,
219     chaos_key: Optional[bytes] = None,
220     precomputed_positions: Optional[list[tuple[int, int, int]]] = None,
221     trust_precomputed_positions: bool = False,
222     use_analysis_cache: bool = True,
223     force_analysis_refresh: bool = False,
224     analysis_cache_dir: Optional[str] = None,
225     precomputed_analysis: Optional[tuple] = None,
226 ) -> EmbedResult:
227     """
228     Embed a Groth16 ZK proof for `message` into an H.264 video.
229
230     Pipeline:
231     1. Generate ZK proof (snarkjs Groth16, via Node.js)
232     2. Pack payload blob [4B len][message][129B proof]
233     2b. [Chaos] Arnold Cat Map scrambles payload bits (if chaos_key)
234     3. Parse H.264 video extract IDR coefficients + bit offsets
235     4. Safety filter find safe trailing-ones positions
236     4b. [Chaos] Logistic Map shuffles embedding positions (if chaos_key)
237     5. Embed payload flip T1 sign bits
238     6. Reconstruct video patch bitstream at tracked offsets
239
240     Args:
241     video_path: Input H.264 video path.
242     message: Secret message bytes to commit to.
243     output_path: Output stego H.264 video path.

```

```

244     circuits_dir: Path to circuits/ directory (ZK build artifacts).
245     secret_key: 32-byte secret key (never embedded, used for commitment).
246     max_modifications_per_block: T1 flips per 4x4 block (default 1).
247     ffmpeg_validate: Enable per-position Ffmpeg pixel validation.
248                     Slower but guarantees no visible artefacts.
249     chaos_key: Optional bytes key to enable chaos math transforms.
250                When set, Arnold Cat Map scrambles payload bits and
251                Logistic Map shuffles the embedding position order,
252                making the hidden data resist steganalysis.
253                Pass the same key to verify() for correct extraction.
254     precomputed_positions: Optional externally supplied operating positions.
255                When provided, these are used directly and bypass the
256                default safe-position ordering / pruning flow.
257     trust_precomputed_positions: When True, keep externally supplied
258                operating positions as the authoritative bit budget
259                instead of re-inferring usable bits from applied_block_keys.
260                Intended for benchmark-grade operating points that have
261                already been validated end-to-end.
262     use_analysis_cache: Reuse cached cover-video analysis when available.
263                Strongly recommended for app/runtime usage.
264     force_analysis_refresh: Ignore cached cover analysis and rebuild it.
265     analysis_cache_dir: Optional custom directory for analysis cache files.
266     precomputed_analysis: Existing immutable video analysis to reuse. This
267                prevents a second full-video cache load in callers
268                which already derived an embedding schedule.
269
270 Returns:
271     EmbedResult
272
273 Raises:
274     FileNotFoundError: If video_path or circuits_dir does not exist.
275     ValueError: If message is empty or secret_key is not 32 bytes.
276     RuntimeError: If video has no IDR frames or capacity is insufficient.
277
278 """
279 # --- Input validation ---
280 if not os.path.isfile(video_path):
281     raise FileNotFoundError(f"Input video not found: {video_path}")
282 if not os.path.isdir(circuits_dir):
283     raise FileNotFoundError(f"circuits_dir not found: {circuits_dir}")
284 if not isinstance(message, bytes) or len(message) == 0:
285     raise ValueError("message must be non-empty bytes")
286 if not isinstance(secret_key, bytes) or len(secret_key) != 32:
287     raise ValueError("secret_key must be exactly 32 bytes")
288 if max_modifications_per_block < 1 or max_modifications_per_block > 8:
289     raise ValueError("max_modifications_per_block must be between 1 and 8")
290 if ffmpeg_validate and shutil.which("ffmpeg") is None:
291     raise RuntimeError(
292         "ffmpeg_validate=True but 'ffmpeg' was not found on PATH."
293     )
294 stream_profile = analyze_stream_profile(video_path)
295 if not stream_profile.is_all_intra:
296     logger.warning(
297         "[Embed] Stream classified as %s; current strongest operating regime remains all-intra H.264/CAVLC",
298         stream_profile.inferred_gop_class,
299     )
300
301 # 1. Generate ZK proof
302 bridge = _get_bridge(circuits_dir)
303 proof_dict, public_dict = bridge.generate_proof_for_payload(message, secret_key)
304 proof_bytes = bridge.proof_to_bytes(proof_dict)
305
306 # 2. Pack payload blob [4B Len][message][129B compressed proof]
307 payload_blob = pack(message, proof_bytes)
308 original_bit_count = len(payload_blob) * 8
309
310 # 2b. Chaos: Arnold Cat Map - scramble payload bits
311 chaos: Optional[ChaosTransformer] = None
312 if chaos_key is not None:
313     chaos = ChaosTransformer(chaos_key)
314     payload_blob, _orig_bits = chaos.scramble(payload_blob)
315     logger.info(
316         "[Chaos] Arnold Cat Map applied: %d bits → %d bits (k=%d)",
317         original_bit_count, len(payload_blob) * 8, chaos.arnold_k,
318     )
319
320 # 3. Parse H.264 + 4. safety filter (runtime cacheable cover analysis)
321 (
322     coefficients,
323     frame_verified_data,
324     nC_map,
325     nal_length_map,
326     t1_override_map,
327     safe_positions,
328 ) = _resolve_video_analysis(
329     video_path,
330     use_analysis_cache=use_analysis_cache,
331     force_analysis_refresh=force_analysis_refresh,
332     analysis_cache_dir=analysis_cache_dir,
333     precomputed_analysis=precomputed_analysis,
334 )
335 raw_safe_bits = len(safe_positions)

```

```

335 ffmpeg_validated_bits: Optional[int] = None
336 patchable_usable_bits: Optional[int] = None
337 requested_position_bits: Optional[int] = None
338
339 if precomputed_positions is not None:
340     safe_positions = _limit_positions_per_block(
341         [tuple(int(v) for v in pos) for pos in precomputed_positions],
342         max_modifications_per_block=max_modifications_per_block,
343     )
344     requested_position_bits = len(safe_positions)
345 else:
346     # 4b. Chaos: Logistic Map - shuffle position order
347     if chaos is not None:
348         safe_positions = chaos.shuffle_positions(safe_positions)
349         logger.info(
350             "[Chaos] Logistic Map shuffle applied to %d positions (seed=%.6f)",
351             len(safe_positions), chaos.logistic_seed,
352         )
353
354     # 4c. Optional Ffmpeg validator - pre-filter positions BEFORE passing as
355     # pre_validated_positions. stego.py sets ffmpeg_validator=None whenever
356     # pre_validated_positions is provided, so we must validate here instead.
357     # Cap at 5x required bits to prevent hour-long loops on large position pools.
358     if ffmpeg_validate:
359         rec = BitstreamReconstructor()
360         _vfn, _cleanup = rec.make_ffmpeg_position_validator(
361             video_path, coefficients, frame_verified_data
362         )
363         _required = len(payload_blob) * 8
364         _max_tries = max(_required * 5, 2000)
365         _total_candidates = len(safe_positions)
366         _validated: list = []
367         _tried = 0
368         for p in safe_positions:
369             if len(_validated) >= _required or _tried >= _max_tries:
370                 break
371             _tried += 1
372             if _vfn(p[0], p[1], p[2]):
373                 _validated.append(p)
374         _cleanup()
375         logger.info(
376             "[Ffmpeg] %d/%d passed (tried %d/%d candidates, needed %d)",
377             len(_validated), _tried, _tried, _total_candidates, _required,
378         )
379         safe_positions = _validated
380         ffmpeg_validated_bits = len(safe_positions)
381
382     # 4d. Keep only positions whose blocks are patchable in the original bitstream.
383     required_positions = len(payload_blob) * 8
384     headroom_positions = max(required_positions + 256, int(required_positions * 1.30))
385     safe_positions = _prune_patchable_positions(
386         safe_positions,
387         frame_verified_data,
388         required_bits=headroom_positions,
389         max_modifications_per_block=max_modifications_per_block,
390     )
391     safe_positions = _limit_positions_per_block(
392         safe_positions,
393         max_modifications_per_block=max_modifications_per_block,
394     )
395     patchable_usable_bits = len(safe_positions)
396
397 if ffmpeg_validated_bits is None and precomputed_positions is None:
398     ffmpeg_validated_bits = len(safe_positions)
399 if patchable_usable_bits is None:
400     patchable_usable_bits = len(safe_positions)
401 if requested_position_bits is None:
402     requested_position_bits = len(safe_positions)
403
404 required_bits = len(payload_blob) * 8
405
406 # 6. Reconstruct stego video
407 os.makedirs(os.path.dirname(os.path.abspath(output_path)), exist_ok=True)
408 reconstruction_context = load_or_build_reconstruction_context(
409     video_path,
410     use_cache=use_analysis_cache,
411     force_refresh=force_analysis_refresh,
412     cache_dir=analysis_cache_dir,
413 )
414 blocked_reconstruct_blocks: set[tuple[int, int]] = set()
415 used_positions: list[tuple[int, int, int]] = []
416 bits_embedded = 0
417 applied_position_bits = 0
418 for _attempt in range(12):
419     candidate_positions = [
420         (int(mb), int(blk), int(cidx))
421         for mb, blk, cidx in safe_positions
422         if (int(mb), int(blk)) not in blocked_reconstruct_blocks
423     ]
424
425 embedder = PayloadEmbedder(max_modifications_per_block=max_modifications_per_block)

```

```

426         modified, bits_embedded = embedder.embed_payload(
427             coefficients, payload_blob,
428             nC_map=nC_map,
429             nal_length_map=nal_length_map,
430             t1_override_map=t1_override_map,
431             frame_verified_data=frame_verified_data,
432             ffmpeg_validator=None,
433             pre_validated_positions=candidate_positions,
434         )
435
436         if bits_embedded < required_bits:
437             raise InsufficientCapacityError(
438                 required_bits=required_bits,
439                 available_bits=bits_embedded,
440                 stage="pre_reconstruct_embedding",
441                 raw_safe_bits=raw_safe_bits,
442                 ffmpeg_validated_bits=ffmpeg_validated_bits,
443                 patchable_usable_bits=patchable_usable_bits,
444                 requested_position_bits=requested_position_bits,
445                 trust_precomputed_positions=trust_precomputed_positions,
446                 chaos_enabled=bool(chaos is not None),
447             )
448
449         used_positions = [
450             (int(mb), int(blk), int(cidx))
451             for mb, blk, cidx in getattr(embedder, "last_used_safe_positions", [])
452         ]
453         modified_block_keys = {
454             (int(mb), int(blk))
455             for mb, blk, _coeffs in modified
456         }
457
458         rec2 = BitstreamReconstructor()
459         reconstruction_stats = rec2.reconstruct_video(
460             video_path, modified, output_path,
461             max_slices=None,
462             frame_verified_data=frame_verified_data,
463             reconstruction_context=reconstruction_context,
464         )
465
466         applied_block_keys = {
467             (int(mb), int(blk))
468             for mb, blk in reconstruction_stats.get("applied_block_keys", [])
469         }
470         filtered_positions, missing_modified_blocks = _filter_reconstructed_positions(
471             used_positions=used_positions,
472             modified_block_keys=modified_block_keys,
473             applied_block_keys=applied_block_keys,
474         )
475         if len(filtered_positions) >= required_bits:
476             used_positions = filtered_positions[:required_bits]
477             bits_embedded = len(used_positions)
478             applied_position_bits = len(used_positions)
479             break
480
481         blocked_reconstruct_blocks.update(missing_modified_blocks)
482         used_positions = filtered_positions
483         bits_embedded = len(used_positions)
484         applied_position_bits = len(used_positions)
485         if not missing_modified_blocks:
486             break
487
488         if bits_embedded < required_bits:
489             raise InsufficientCapacityError(
490                 required_bits=required_bits,
491                 available_bits=bits_embedded,
492                 stage="post_reconstruct_application",
493                 raw_safe_bits=raw_safe_bits,
494                 ffmpeg_validated_bits=ffmpeg_validated_bits,
495                 patchable_usable_bits=patchable_usable_bits,
496                 requested_position_bits=requested_position_bits,
497                 applied_position_bits=applied_position_bits,
498                 trust_precomputed_positions=trust_precomputed_positions,
499                 chaos_enabled=bool(chaos is not None),
500             )
501
502         # Save positions.json (legacy, for compatibility)
503         pos_path = f"{output_path}.positions.json"
504         with open(pos_path, "w", encoding="utf-8") as f:
505             json.dump([[mb, blk, cidx] for mb, blk, cidx in used_positions], f, ensure_ascii=True, indent=2)
506
507         # Save meta.json (legacy, for compatibility)
508         meta_path = f"{output_path}.meta.json"
509         with open(meta_path, "w", encoding="utf-8") as f:
510             json.dump(
511                 {
512                     "bits_embedded": int(bits_embedded),
513                     "bits_required": int(required_bits),
514                     "positions_count": len(used_positions),
515                     "raw_safe_bits": raw_safe_bits,
516                     "ffmpeg_validated_bits": ffmpeg_validated_bits,

```

```

517         "patchable_usable_bits": patchable_usable_bits,
518         "requested_position_bits": requested_position_bits,
519         "applied_position_bits": applied_position_bits,
520         "chaos_enabled": bool(chaos is not None),
521         "chaos_original_bits": (int(original_bit_count) if chaos is not None else None),
522     },
523     f,
524     ensure_ascii=True,
525     indent=2,
526 )
527
528 # Save versioned manifest.json
529 manifest = StegoManifest(
530     payload=PayloadMetadata(
531         message_length=len(message),
532         bits_embedded=bits_embedded,
533         bits_required=required_bits,
534         chaos_enabled=chaos is not None,
535         chaos_original_bits=original_bit_count if chaos is not None else None,
536         chaos_expansion_factor=len(payload_blob) * 8 / original_bit_count if chaos is not None else 1.0,
537     ),
538     embedding=EmbeddingMetadata(
539         strategy="t1_sign_flip",
540         max_modifications_per_block=max_modifications_per_block,
541         positions_count=len(used_positions),
542     ),
543     video=VideoMetadata(
544         file_path=video_path,
545         file_hash=compute_file_hash(video_path),
546         codec="h264",
547         profile="baseline",
548     ),
549     proof=ProofMetadata(
550         proof_system="groth16",
551         proof_size_bytes=len(proof_bytes),
552         constraint_count=bridge.get_constraint_count(),
553     ),
554 )
555 manifest_path = f"{output_path}.manifest.json"
556 manifest.save(manifest_path)
557 capacity = sum(
558     1 for _, _, coeffs in coefficients for v in coeffs if abs(v) == 1
559 )
560
561 return EmbedResult(
562     bits_embedded=bits_embedded,
563     capacity_bits=capacity,
564     stream_class=stream_profile.inferred_gop_class,
565     raw_safe_bits=raw_safe_bits,
566     patchable_usable_bits=patchable_usable_bits,
567     ffmpeg_validated_bits=ffmpeg_validated_bits,
568     requested_position_bits=requested_position_bits,
569     applied_position_bits=applied_position_bits,
570     output_path=output_path,
571     proof_dict=proof_dict,
572     public_dict=public_dict,
573     chaos_original_bits=original_bit_count if chaos is not None else None,
574     used_positions=used_positions,
575 )

```

Nhận xét review. Đây là toàn bộ embed() hiện tại, không phải bản tóm tắt. Từ đầu đến cuối hàm validate path/key/message; profile H.264; tạo proof; pack; resolve cache/precomputed analysis; lọc safe/patchable candidates; nhúng payload qua PayloadEmbedder; reconstruct NAL/slice; lọc positions thật sự áp dụng; ghi output/sidecars và trả EmbedResult. Handoff quan trọng là candidate schedule và reconstructed video phải cùng phản ánh patch thực tế. Sidecar positions thuộc nhánh này; chúng không phải format bắt buộc của native blind route.

8.2 Verifier của Python route

Listing 17: Python source, dòng 97–248

```

97 -
98 def verify(
99     stego_video_path: str,
100     original_video_path: str,
101     circuits_dir: str,
102     secret_key: bytes,
103     message_length: int,
104     max_modifications_per_block: int = 1,
105     chaos_key: Optional[bytes] = None,
106     precomputed_positions: Optional[list[tuple[int, int, int]]] = None,
107     precomputed_payload_bits: Optional[int] = None,
108     use_analysis_cache: bool = True,

```

```

109     force_analysis_refresh: bool = False,
110     analysis_cache_dir: Optional[str] = None,
111 ) -> VerifyResult:
112     """
113     Extract and verify the ZK proof embedded in a stego H.264 video.
114
115     Pipeline:
116     1. Parse original video - reconstruct embedding position map
117     2. Get safe positions - same order as used during embed
118     2b. [Chaos] Logistic Map shuffle positions (if chaos_key)
119     3. Extract bits - read T1 signs from stego video
120     3b. [Chaos] Arnold Cat Map unscramble payload bits (if chaos_key)
121     4. Unpack blob - split into (message, proof_bytes)
122     5. Verify ZK proof - snarkjs groth16 verify
123
124     Args:
125     stego_video_path: Path to the stego H.264 video.
126     original_video_path: Path to the original (pre-embed) H.264 video.
127     circuits_dir: Path to circuits/ directory (ZK build artifacts).
128     secret_key: 32-byte secret key used during embedding.
129     message_length: Expected message length in bytes.
130     max_modifications_per_block: Must match value used in embed() (default 1).
131     chaos_key: Must match the chaos_key passed to embed() - if embed()
132               was called with chaos_key, pass the same value here.
133     precomputed_positions: Optional final embedding positions to use directly.
134                           When provided, these must already reflect any chaos
135                           shuffle and benchmark-side validation/pruning.
136     precomputed_payload_bits: Optional exact number of payload bits to extract.
137                              Use with precomputed_positions when the benchmark
138                              embedded a chaos-padded payload whose size differs
139                              from blob_bit_length(message).
140     use_analysis_cache: Reuse cached cover-video analysis when available.
141     force_analysis_refresh: Ignore cached cover analysis and rebuild it.
142     analysis_cache_dir: Optional custom directory for analysis cache files.
143
144     Returns:
145     VerifyResult with valid flag, message, and proof details.
146
147     Raises:
148     FileNotFoundError: If stego_video_path, original_video_path, or circuits_dir does not exist.
149     ValueError: If secret_key is not 32 bytes or message_length is not positive.
150     """
151     # --- Input validation ---
152     if not os.path.isfile(stego_video_path):
153         raise FileNotFoundError(f"Stego video not found: {stego_video_path}")
154     if not os.path.isfile(original_video_path):
155         raise FileNotFoundError(f"Original video not found: {original_video_path}")
156     if not os.path.isdir(circuits_dir):
157         raise FileNotFoundError(f"Circuits dir not found: {circuits_dir}")
158     if not isinstance(secret_key, bytes) or len(secret_key) != 32:
159         raise ValueError("secret_key must be exactly 32 bytes")
160     if message_length <= 0:
161         raise ValueError("message_length must be a positive integer")
162
163     if precomputed_positions is None or precomputed_payload_bits is None:
164         sidecar_positions, sidecar_bits, _ = _load_sidecar_data(stego_video_path)
165         if precomputed_positions is None and sidecar_positions is not None:
166             precomputed_positions = sidecar_positions
167         if precomputed_payload_bits is None and sidecar_bits is not None:
168             precomputed_payload_bits = sidecar_bits
169
170     # 1. Parse original video + 2. recover safe positions (runtime cacheable)
171     (
172         coefficients,
173         frame_verified_data,
174         nC_map,
175         nal_length_map,
176         t1_override_map,
177         safe_positions,
178     ) = load_or_build_video_analysis(
179         original_video_path,
180         use_cache=use_analysis_cache,
181         force_refresh=force_analysis_refresh,
182         cache_dir=analysis_cache_dir,
183     )
184
185     # 2b. Chaos: Logistic Map - apply the same position shuffle as embed()
186     chaos: Optional[ChaosTransformer] = None
187     original_bit_count = blob_bit_length(b"\x00" * message_length)
188     if precomputed_positions is not None:
189         safe_positions = [tuple(int(v) for v in pos) for pos in precomputed_positions]
190         if chaos_key is not None:
191             chaos = ChaosTransformer(chaos_key)
192     elif chaos_key is not None:
193         chaos = ChaosTransformer(chaos_key)
194         safe_positions = chaos.shuffle_positions(safe_positions)
195         logger.info(
196             "[Chaos] Logistic Map shuffle applied to %d positions",
197             len(safe_positions),
198         )
199

```

```

200 # 3. Compute how many bits to extract
201 if precomputed_payload_bits is not None:
202     extract_bit_count = int(precomputed_payload_bits)
203 elif chaos is not None:
204     # Must extract M^2 bits (padded) to correctly unscramble
205     padded_bits = ChaosTransformer.padded_bit_count(original_bit_count)
206     # Round up to byte boundary for extract_bits_direct
207     extract_bit_count = math.ceil(padded_bits / 8) * 8
208 elif precomputed_positions is not None:
209     extract_bit_count = len(safe_positions)
210 else:
211     extract_bit_count = original_bit_count
212
213 # 3a. Extract bits from stego video
214 extracted_blob = extract_bits_direct(
215     stego_video_path,
216     safe_positions,
217     frame_verified_data,
218     nC_map,
219     extract_bit_count,
220     max_modifications_per_block=max_modifications_per_block,
221 )
222
223 # 3b. Chaos: Arnold Cat Map - unscramble extracted bits
224 if chaos is not None:
225     extracted_blob = chaos.unscramble(extracted_blob, original_bit_count)
226     logger.info("[Chaos] Arnold Cat Map inverse applied (k=%d)", chaos.arnold_k)
227
228 # 4. Unpack blob -> (message, proof_bytes)
229 try:
230     message, proof_bytes = unpack(extracted_blob)
231 except ValueError:
232     return VerifyResult(
233         valid=False, message=None, proof_dict=None,
234         public_dict=None, bits_extracted=len(extracted_blob) * 8,
235     )
236
237 # 5. Verify ZK proof
238 bridge = _get_bridge(circuits_dir)
239 proof_dict = bridge.bytes_to_proof(proof_bytes)
240 public_signals = bridge._build_public_signals(message, secret_key)
241 is_valid = bridge.verify(proof_dict, public_signals)
242
243 return VerifyResult(
244     valid=is_valid,
245     message=message if is_valid else None,
246     proof_dict=proof_dict,
247     public_dict=public_signals,
248     bits_extracted=extract_bit_count,
249 )

```

Nhận xét review. Toàn bộ `verify()` cho thấy verifier Python nhận cả stego và original video, hoặc position metadata được chuẩn bị trước. Nó dựng schedule/số bit, trích coefficients, unpack proof, tái dựng public inputs và gọi `bridge.verify()`. Vì phụ thuộc original cover/sidecar, không được quảng bá hàm này như blind extractor native. Output `VerifyResult` cần đọc cùng các trường `valid/message/bits_extracted`; exception `parse/capacity` khác với cryptographic `false`.

8.3 Python H.264/CAVLC parser và safety layer

Nhánh Python tổ chức parser thành các lớp: NAL parser, slice-header parser, macroblock parser và CAVLC decoder. Sau đó parser traceable tạo bit offsets cho safety filter. Trong `src/core/stego.py`, `PayloadEmbedder.embed\_payload()` sửa coefficient sign/LSB; patcher/reconstructor tái mã hóa slice và viết lại file. Đường này khác native ở representation, schedule và reconstruction strategy; không được xem code Python là wrapper mỏng quanh C++.

Nhận xét review. Python route cần kiểm tra lỗi bit-length drift, coefficient level/run reconstruction và NAL emulation-prevention khi viết lại. Safety filter chỉ xác định các ứng viên theo invariant đã code; chỉ strict decode/visual metric ở output mới cho biết stream còn giải mã/hiển thị ra sao. “Parser không lỗi” không tương đương PSNR/SSIM không đổi.

9 Trust, key policy và provenance: module hỗ trợ, không tự động cưỡng chế

`src/key_policy.py` có thể issue/verify certificate với cửa sổ thời gian; `src/manifest.py` ký metadata; `src/trust/` chứa fingerprint/attestation/provenance helpers. Các chức năng này chỉ có hiệu lực khi verifier/application thật sự gọi chúng ở mọi điểm cấp/chấp nhận key và artifact.

Nhận xét review. Hiện native demo/API truyền raw 32-byte key trực tiếp cho native CLI; nó không tự đọc certificate/expiry. Vì vậy không được kết luận “key tự hết hạn” chỉ vì module policy tồn tại. Thời hạn cần được enforce tại service boundary: xác thực certificate, kiểm đồng hồ/counter, từ chối key hết hạn, đồng thời có quy trình rotate/revoke. Bearer API token và key embed là hai credential khác nhau.

10 Contract giữa các bước: input/output và lỗi cần phân loại

Boundary	Input	Output	Lỗi riêng biệt
ZK bridge	message bytes + 32-byte key + setup artifacts	proof dictionary + public signals	Setup/tool failure khác proof invalid.
Serializer	proof dictionary	fixed proof bytes	Serialization/curve validation error.
Payload packer	message + proof bytes	[4-byte length][message][proof]	Malformed length/trailing bytes.
Native encoder	Annex-B + key + payload	Stego Annex-B + capacity/complete	Unsupported syntax/capacity exhaustion.
Native decoder	Stego Annex-B + same key + max length	authenticated payload bytes	Wrong key, invalid tag/version or truncated stream.
Unpacker	payload bytes	message + proof bytes	Format invalid/truncated proof.
Groth16 verifier	proof + public signals + verification key	Boolean result	Invalid proof khác extraction failure.
API job layer	HTTP upload + bearer + fields	job id/status/artifact	HTTP 401, 413, 429, 501 khác pipeline error.

11 Kết luận review: hệ thống hiện có và chưa có gì

Hiện source chứa một native CAVLC blind channel có authenticated framing, CLI incremental và adapter HTTP/WebSocket; một lớp Python Groth16 để tạo/xác minh proof; cùng pipeline Python full-video cũ với sidecar/reconstruction. Đây là ba trách nhiệm liên kết qua payload format, không phải một hàm duy nhất.

Không được suy ra rằng: chỉ gửi video là đủ để verifier chạy; HTTP native endpoint đã verify Groth16; mọi H.264 profile đều hỗ trợ; CAVLC/HMAC frame thay thế ZK; certificate helper tự enforce expiry; hoặc successful decode tương đương visual quality tốt. Các ranh giới cần được test riêng: format proof, CAVLC parser schedule, payload/frame auth, actual video decode/visual metrics, và Groth16 public-input verification.